

# Physics-informed sparse identification-based tube model predictive control for aerial vehicles

Tayyab Manzoor<sup>a</sup>, Yasir Ali<sup>b,c</sup>, Yuanqing Xia<sup>d,b,\*</sup>, Lijie You<sup>a,b</sup>, Yan Wang<sup>a</sup>

<sup>a</sup>*School of Automation and Electrical Engineering, Zhongyuan University of Technology, Zhengzhou, Henan 450007, China*

<sup>b</sup>*School of Automation, Beijing Institute of Technology, Beijing 100081, China*

<sup>c</sup>*Zhengzhou Academy of Intelligent Technology, Beijing Institute of Technology, Zhengzhou 450000, China*

<sup>d</sup>*Zhongyuan University of Technology, Zhengzhou, Henan 450007, China*

## Abstract

Autonomous aerial vehicles necessitate control strategies that balance computational efficiency with robust performance in dynamic operational environments. This paper proposes a model predictive control (MPC) framework for aerial platforms that leverages physics-informed machine learning (PIML) to achieve an optimal balance between computational tractability and robust performance. At the core of the proposed approach lies a sparse, control-affine model identified via the PIML method, which provides a parsimonious yet interpretable representation of the system dynamics by embedding first-principles knowledge and learning residual uncertainties from operational data. This model is incorporated within a robust MPC scheme that adopts a high-order Runge-Kutta discretization to ensure prediction accuracy and an adaptive tube-based mechanism to guarantee constraint satisfaction under uncertainty. The online adaptation of the tube, directly informed by the residual error of the PIML model, ensures robust stability without introducing excessive conservatism. Rigorous theoretical proofs are provided to establish recursive feasibility and stability. Numerical simulations and experiments on a quadrotor demonstrate that our method significantly reduces computational load compared to nonlinear MPC and robust MPC using a first-principles high-fidelity model, while outperforming PID, nonlinear MPC, neural-network-based MPC, and fixed-tube robust MPC in tracking performance and robustness, showcasing the practical efficiency of the proposed PIML-based control synthesis for resource-constrained aerial systems.

**Keywords:** Model predictive control, physics-informed machine learning, sparse regression, unmanned aerial vehicles.

## Nomenclature

|                            |                                        |                                   |                                               |
|----------------------------|----------------------------------------|-----------------------------------|-----------------------------------------------|
| $\bullet^B, \bullet^I$     | Body-fixed and inertial frame          | $\bullet(j/k)$                    | $\bullet$ at $j$ predicted at time $k$        |
| $\hat{\bullet}, \bullet^*$ | Learned and optimal value of $\bullet$ | $\oplus, \ominus$                 | Minkowski sum, Pontryagin difference          |
| $\mathbb{N}, \mathbb{R}$   | Set of natural and real numbers        | $s_\bullet, c_\bullet, t_\bullet$ | $\sin(\bullet), \cos(\bullet), \tan(\bullet)$ |
| $\ \bullet\ $              | Euclidean norm                         | $\mathcal{D}, \mathbb{D}$         | Learning dataset, disturbance set             |
| $\ \bullet\ _Q$            | Q-weighted matrix norm                 | $\mathcal{U}_\xi$                 | Learning-induced uncertainty set              |

## 1. Introduction

Aerial vehicles are emerging as transformative technologies, revolutionizing transportation, industrial automation, and smart urban development. Their applications in autonomous logistics, precision agriculture, and infrastructure

\*Corresponding Author

Email address: xia\_yuanqing@bit.edu.cn (Yuanqing Xia)

inspection are driving significant advancements, reshaping industries, and enabling innovative solutions to complex challenges [1–3]. However, despite their numerous advantages, the widespread adoption of drones presents several challenges. While widely used PID controllers are effective for simple, single-objective tasks like stabilizing an aerial vehicle’s altitude or position, they lack the predictive, adaptive, and collaborative capabilities required for complex tasks [4]. Despite their potential, Machine Learning (ML) techniques alone are insufficient because they often lack the predictive capabilities and real-time adaptability required for dynamic environments, as their purely data-driven nature lacks the fundamental physical constraints and first-principles reasoning necessary for reliable prediction and adaptation. Model predictive control (MPC), on the other hand, helps to mitigate several issues associated with non-optimal control methods [5].

However, MPC law is intrinsically tied to the accuracy of the underlying dynamic model for model-based optimization. Traditionally, controllers for aerial vehicles are developed using first-principles models derived from physics-informed models (PIMs) [6]. While physically intuitive, these nominal models are often insufficiently precise, and they fail to capture complex, hard-to-model effects. This model mismatch manifests as persistent disturbances that can degrade tracking performance, violate safety constraints, and, in worst-case scenarios, lead to system instability. The field of ML offers powerful tools for learning residual dynamics from operational data by embedding PIM, leading to the paradigm of Physics-Informed Machine Learning (PIML) [7]. Several works have successfully demonstrated the use of PIML involving neural network (NN)-based models for model identification [8–15]. Nevertheless, NN-based methods require large training data and are generally not interpretable [16, 17]. Furthermore, another challenge remains in how to certify robust constraint satisfaction and stability for an MPC controller based on a learned model that inherently possesses residual uncertainty. This challenge can be addressed by implementing hybrid control algorithms that leverage the capabilities of ML and MPC techniques [18–20], enabling more robust, adaptive, and precise drone operations in complex environments. To address the issue associated with NN-based control approaches, the sparse identification of nonlinear dynamics (SINDy) driven MPC control technique can be adopted [6, 16, 21–23]. While promising, generally, SINDy techniques suffer due to the manual library selection. Moreover, real-world adoption of PIML techniques for aerial vehicles faces critical challenges in real-time implementation, generalization, data scarcity, interpretability, and computational efficiency. Robust MPC (RMPC) frameworks are designed to handle bounded uncertainties [24, 25]. A common approach is tube-based MPC [26–29], which uses a nominal prediction model and a feedback controller to confine the actual state within a robust positively invariant (RPI) tube around the nominal trajectory. The key to this method is the accurate characterization of the disturbance set, which bounds the discrepancy between the nominal model and the true system. Most existing RMPC approaches assume that the disturbance set is known a priori and constant. This is a significant limitation when using a learned model, as the model error is not constant; it is initially large, decreases as more data is collected, and may vary with the operating regime. A non-adaptive, overly conservative disturbance bound would lead to poor performance, while an optimistic one would jeopardize robustness.

To bridge this gap, this paper proposes a PIML-based tube MPC framework for aerial vehicles that addresses key challenges for PIML-based MPC for aerial vehicles, shown in Fig. 1. Our core contribution is a methodology that seamlessly leverages online model learning with robust control, providing formal guarantees for systems in which the model is refined and its uncertainty quantified in real time. As summarized in Table 1, all existing SINDy-based MPC frameworks for aerial vehicles rely on manually predefined function libraries and do not integrate any tube-based robust constraint mechanism. Different from these works, the proposed method realizes fully automatic library construction and sparse pruning for SINDy identification. More importantly, we combine the sparse learning model with a real-time adaptive tube scheme, which is absent in current SINDy-MPC studies. In addition, we provide complete theoretical guarantees for the overall learning-control system and conduct both numerical simulations and real hardware experiments, further distinguishing our work from existing approaches. In other words, the contributions of this work are threefold:

### 1.1. Contributions

1. Unlike NN-based PIML approaches [8–15], that require large datasets and lack interpretability, we employ sparse regression to learn a compact, physically consistent residual model. This balances accuracy with real-time computational feasibility. Contrary to traditional SINDy-based MPC methods that rely on manually designed function libraries and fixed dictionary structures [6, 16, 21–23], the proposed framework realizes fully

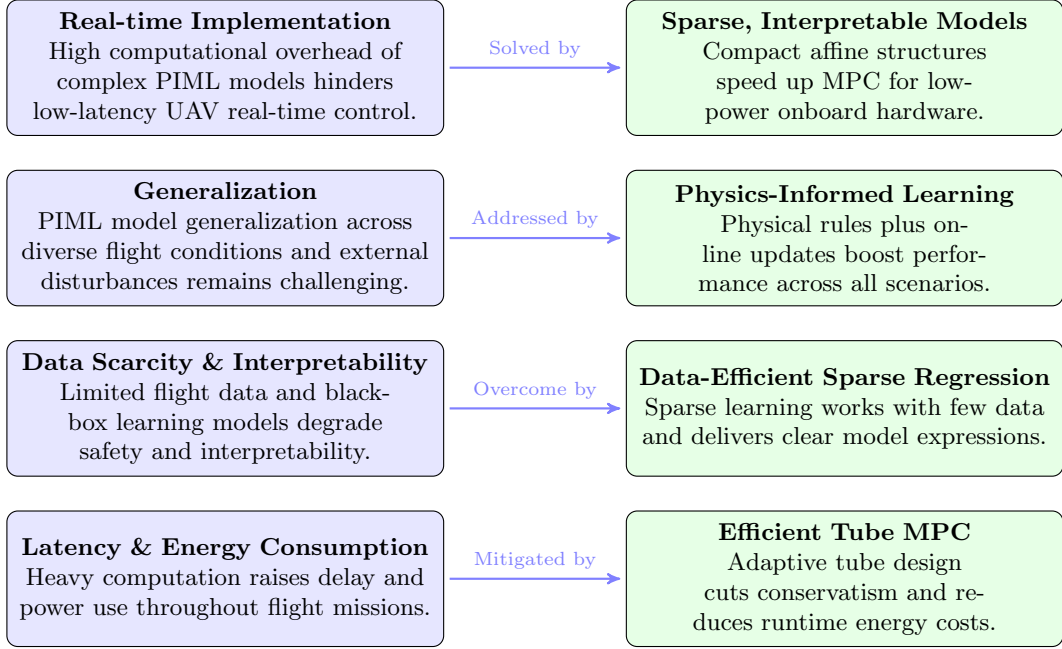


Figure 1: Mapping of key challenges in PIML-based MPC for aerial vehicles to our proposed solutions. Each challenge (left) is directly addressed by a specific contribution (right), demonstrating how our framework overcomes practical deployment barriers.

automatic library initialization, nonlinear expansion, and sparse pruning based on quadrotor rigid-body mechanics. It eliminates artificial intervention in library construction and improves model adaptability while retaining the interpretability and data efficiency of sparse identification. This fundamentally differs from both conventional manually tuned SINDy-MPC and pure data-driven learning-based MPC.

2. Many existing tube-based MPCs adopt constant or empirically preset disturbance sets, leading to excessive conservatism or insufficient robustness. In this work, the disturbance bound is recursively updated using real-time model residual errors and external disturbance measurements, so the tube size adapts to time-varying model uncertainty and wind turbulence. This design breaks the limitation of static bounds in traditional robust tube MPC.
3. While many PIML-based MPC methods lack stability proofs under online adaptation [22, 23], we provide rigorous recursive feasibility and Input-to-State Stability (ISS) guarantees for the closed-loop system, ensuring safety and stability throughout the learning process. We also provide numerical simulations and experimental results, as opposed to [6, 16, 22, 23], to corroborate the efficacy of the developed approach.
4. Different from existing SINDy-MPC with high computational overhead and NN-MPC with large memory consumption, the periodic sparse regression and low-complexity RPI set update ensure the proposed method can run stably on low-power STM32 microcontrollers, achieving a good trade-off among tracking accuracy, robustness, and computational cost.

The remainder of this manuscript is organized as follows. Section 2 formulates the problem with details of the PIML dynamics identification process and control methodology. Section 3 is dedicated to the theoretical analysis, proving recursive feasibility and stability. Section 4 presents numerical simulation results and experiments demonstrating the efficacy of the proposed approach. Finally, Section 5 provides a summary of key findings and suggests avenues for future research.

Table 1: Comparison of existing SINDy-based MPC methods and the proposed approach for aerial vehicles

| Method                  | Description (Model Learning, Tube Adaptation, Theoretical Proof, Real-time Implementation)                                                                                               |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RMPC [6]                | Adaptive SINDy with fixed library; no tube structure; stability and feasibility proofs; simulation-only validation.                                                                      |
| RMPC [16]               | Offline SINDy combined with RL (static library); no tube structure; stability and feasibility proofs; simulation-only validation.                                                        |
| Lyapunov-based MPC [21] | Adaptive SINDy for coefficient update (fixed library); no tube structure; Lyapunov stability proof; validated via both simulation and real-time experiments.                             |
| Conventional MPC [22]   | Static physics-guided SINDy library with offline identification; no tube structure; only model accuracy analysis (no proofs); simulation-only validation.                                |
| Standard MPC [23]       | Fixed manual SINDy library without automatic pruning; no tube structure; no theoretical proofs; validated only via simulation.                                                           |
| Proposed Method         | Physics-informed automatic library + online sparse pruning; adaptive tube mechanism; complete stability and feasibility proofs; validated via both simulation and real-time experiments. |

## 2. Problem formulation and control methodology

This section establishes the mathematical foundation for our approach, beginning with the PIML dynamics model and proceeding to the MPC formulation.

### 2.1. PIML dynamics

The derivation of the PIML dynamics model for the aerial vehicle proceeds in two sequential steps. First, the PIM is established, and this foundational step distills the system’s dynamics into a structured analytical framework. As defined in Fig. 2, two coordinate frames are introduced: the body-fixed frame  $\mathcal{B} \triangleq \{o^{\mathcal{B}}, x^{\mathcal{B}}, y^{\mathcal{B}}, z^{\mathcal{B}}\}$  and the inertial frame  $\mathcal{I} \triangleq \{o^{\mathcal{I}}, x^{\mathcal{I}}, y^{\mathcal{I}}, z^{\mathcal{I}}\}$ . The vehicle’s dynamics are governed by a set of differential equations derived from rigid-body mechanics. The PIM encapsulates this first-principles model, defining the state variables, control inputs, and the fundamental structure of the dynamic relationships. This model provides the architectural prior for the subsequent ML component and is provided as follows:

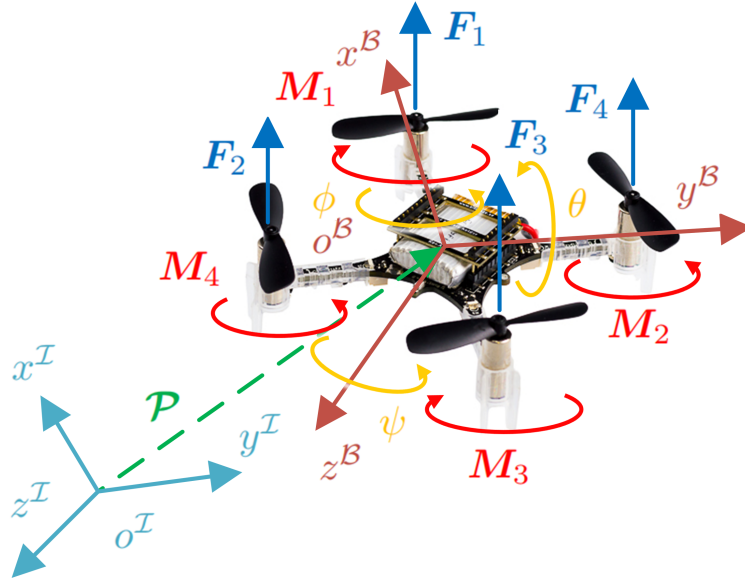


Figure 2: Aerial vehicle’s inertial and body-fixed frame

$$\begin{aligned}
\dot{\mathcal{P}} &= [\dot{\mathcal{P}}_x \ \dot{\mathcal{P}}_y \ \dot{\mathcal{P}}_z]^\top = \mathbf{v}^I = [v_x^I \ v_y^I \ v_z^I]^\top, \\
\dot{\mathbf{v}}^I &= \frac{1}{m} (\underbrace{[0 \ 0 \ mg]^\top}_{\text{Gravity}} + \underbrace{\mathbf{R}^{\mathcal{BI}}(\Theta) [0 \ 0 \ u_1]^\top}_{\text{Thrust}}), \\
\dot{\Theta} &= \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & s_\phi t_\theta & c_\phi t_\theta \\ 0 & c_\phi & -s_\phi \\ 0 & s_\phi/c_\theta & c_\phi/c_\theta \end{bmatrix}}_{\text{Euler angle rate matrix}} \boldsymbol{\omega}^{\mathcal{B}}, \\
\dot{\boldsymbol{\omega}}^{\mathcal{B}} &= \mathbf{J}^{-1} (\underbrace{[u_2 \ u_3 \ u_4]^\top}_{\text{Torque}} - \boldsymbol{\omega}^{\mathcal{B}} \times \mathbf{J} \boldsymbol{\omega}^{\mathcal{B}}),
\end{aligned} \tag{1}$$

with

$$\begin{aligned}
u_1 &= \sum_{i=1}^4 F_i, \quad \mathbf{J} = \text{diag} [J_{xx} \ J_{yy} \ J_{zz}]^\top, \\
\begin{bmatrix} u_2 \\ u_3 \\ u_4 \end{bmatrix} &= \begin{bmatrix} l(F_2 - F_4) \\ l(F_3 - F_1) \\ \kappa(F_1 - F_2 + F_3 - F_4) \end{bmatrix}, \\
\mathbf{R}^{\mathcal{BI}}(\Theta) &= \begin{bmatrix} c_\theta c_\psi & c_\theta s_\psi & -s_\theta \\ s_\phi s_\theta c_\psi - c_\phi s_\psi & s_\phi s_\theta s_\psi + c_\phi c_\psi & s_\phi c_\theta \\ c_\phi s_\theta c_\psi + s_\phi s_\psi & c_\phi s_\theta s_\psi - s_\phi c_\psi & c_\phi c_\theta \end{bmatrix},
\end{aligned}$$

where  $\mathcal{P}$ ,  $\mathbf{v}^I$ ,  $\boldsymbol{\omega}^{\mathcal{B}}$ ,  $m$ ,  $\Theta$  are the position, linear velocity, angular velocity, mass, and attitude of the aerial vehicle, respectively.  $\phi$ ,  $\theta$ ,  $\psi$  are the Euler angles. Moreover,  $s_\bullet = \sin(\bullet)$ ,  $c_\bullet = \cos(\bullet)$ ,  $t_\bullet = \tan(\bullet)$ .  $l$  and  $\kappa$  are arm length and Torque-to-thrust ratio, respectively.  $\Delta_v$  and  $\Delta_\omega$  are the learned parts in the translational and rotational dynamics of the aerial vehicle, respectively. Any deviation in the vehicle dynamics is handled by the ML approach, that is utilized to learn the time-series data, where data is collected, which consists of measurements of the state  $\mathbf{x} = [\mathcal{P}, \boldsymbol{\omega}^{\mathcal{B}}, \mathbf{v}^I, \Theta]^\top$  and input  $\mathbf{u} = [u_1, u_2, u_3, u_4]^\top$ , which includes evaluating the required number of snapshots  $n_s$  of  $\mathbf{x}$ ,  $\mathbf{u}$ . Hence, the dataset  $\mathcal{D}$  can be described in the following two matrices  $\hat{\mathbf{x}}$  and  $\hat{\mathbf{u}}$ .

$$\begin{aligned}
\hat{\mathbf{x}} &= \begin{bmatrix} \mathcal{P}_x^I(t_1) & \mathcal{P}_y^I(t_1) & \cdots & \psi(t_1) \\ \vdots & \vdots & \ddots & \vdots \\ \mathcal{P}_x^I(t_i) & \mathcal{P}_y^I(t_i) & \cdots & \psi(t_i) \\ \vdots & \vdots & \ddots & \vdots \\ \mathcal{P}_x^I(t_{n_s}) & \mathcal{P}_y^I(t_{n_s}) & \cdots & \psi(t_{n_s}) \end{bmatrix}, \\
\hat{\mathbf{u}} &= \begin{bmatrix} u_1(t_1) & u_2(t_1) & u_3(t_1) & u_4(t_1) \\ \vdots & \vdots & \vdots & \vdots \\ u_1(t_i) & u_2(t_i) & u_3(t_i) & u_4(t_i) \\ \vdots & \vdots & \ddots & \vdots \\ u_1(t_{n_s}) & u_2(t_{n_s}) & u_3(t_{n_s}) & u_4(t_{n_s}) \end{bmatrix},
\end{aligned} \tag{2}$$

where  $t_i$  represents the sampling time. Therefore, the numerical differentiation yields the quadrotor's state as follows:

$$\dot{\hat{\mathbf{x}}} = \begin{bmatrix} \dot{\mathcal{P}}_x^I(t_1) & \dot{\mathcal{P}}_y^I(t_1) & \cdots & \dot{\psi}(t_1) \\ \vdots & \vdots & \ddots & \vdots \\ \dot{\mathcal{P}}_x^I(t_i) & \dot{\mathcal{P}}_y^I(t_i) & \cdots & \dot{\psi}(t_i) \\ \vdots & \vdots & \ddots & \vdots \\ \dot{\mathcal{P}}_x^I(t_{n_s}) & \dot{\mathcal{P}}_y^I(t_{n_s}) & \cdots & \dot{\psi}(t_{n_s}) \end{bmatrix}. \tag{3}$$

Thereafter, a library function  $\Psi(\hat{\mathbf{x}}, \hat{\mathbf{u}})$  is constructed using physics-informed rigid-body dynamic terms, including  $\hat{\mathbf{x}}$  and  $\hat{\mathbf{u}}$ , their first-order products, and pairwise multiplicative interactions. No manual customization or domain-specific heuristic terms are introduced, preserving fully automated and interpretable sparse modeling for quadrotor dynamics, which is provided as follows:

$$\Psi(\hat{\mathbf{x}}, \hat{\mathbf{u}}) = [\mathbf{1}^\top, \hat{\mathbf{x}}^\top, (\hat{\mathbf{x}} \otimes \hat{\mathbf{x}})^\top, (\hat{\mathbf{x}} \otimes \hat{\mathbf{u}})^\top, \dots, S_{\hat{\mathbf{x}}^\top}, S_{\hat{\mathbf{u}}^\top}, S_{\hat{\mathbf{x}} \otimes \hat{\mathbf{u}}^\top}, \dots], \quad (4)$$

where  $\otimes$  represents the product combination of the components. Therefore, the sparse model can be expressed as follows:

$$\hat{f}(\hat{\mathbf{x}}, \hat{\mathbf{u}}, S_j) = \sum_{k=0}^{n_{cf}} S_{\ell k} \delta_\ell(\hat{\mathbf{x}}, \hat{\mathbf{u}}), \quad j = 1, 2, 3, 4, \dots, 12, \quad (5)$$

where  $n_{cf}$ ,  $S_{\ell k}$ ,  $\delta_\ell$  denote the number of candidate functions, candidate functions of  $\ell$ -th column in  $\Psi(\hat{\mathbf{x}}, \hat{\mathbf{u}})$  and

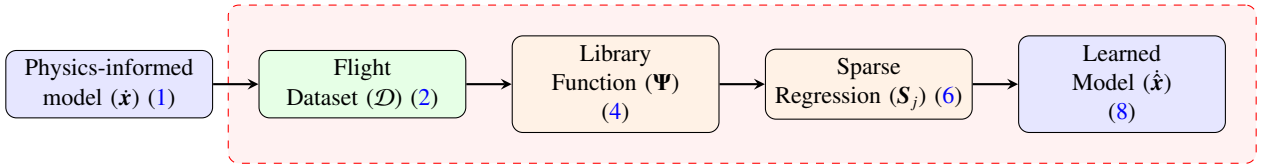


Figure 3: PIML framework

weighted coefficient of  $\delta_\ell(\hat{\mathbf{x}}, \hat{\mathbf{u}})$  concerned with  $j$ -th state, i.e.,  $S_j = [S_{j0}, S_{j1}, S_{j2}, \dots, S_{jn_{cf}}]^\top$ , respectively. Next, sparse optimization is used as follows:

$$S_j = \arg \min_{S_j} \|\hat{\mathbf{x}}_j - S_j \Psi(\hat{\mathbf{x}}, \hat{\mathbf{u}})^\top\|_2 + h_j \|S_j\|_1, \quad (6)$$

where  $\hat{\mathbf{x}}_j$ ,  $h_j$  are the  $j$ -th row of  $\hat{\mathbf{x}}$  represent hyper-parameter, respectively.  $S_k$  is stored in matrix  $\xi$ ,

$$\xi = [S_1 S_2 \dots S_j \dots S_{12}]. \quad (7)$$

Therefore, the learned model can be defined as:

$$\hat{\mathbf{x}} = \hat{f}(\hat{\mathbf{x}}, \hat{\mathbf{u}}) \approx \Psi(\hat{\mathbf{x}}, \hat{\mathbf{u}}) \xi, \quad (8)$$

where  $\hat{f}(\hat{\mathbf{x}}, \hat{\mathbf{u}})$  is the learned model. The complete process is depicted in Fig. 3. The library function selection is an automatic process. The initial library is populated with universal rigid-body mechanics terms (e.g.,  $\hat{\mathbf{x}}$ ,  $\hat{\mathbf{u}}$ , etc.) and is not handpicked but fundamental to modeling quadrotor motion (e.g., position, velocity, control input interactions). Non-linear candidate functions are auto-added when the flight dataset  $\mathcal{D}$  exceeds  $N_{min} = 500$  samples.  $N_{min}$  is empirically tuned via preliminary flight tests to balance noise filtering and computational overhead for auto library expansion. This trigger ensures the library adapts to unmodeled dynamics (e.g., ground effect, wind) without manual intervention, and no user decides which terms to include, so the automatic property of the library is fully preserved. The relevant terms are discarded automatically via regression, leaving only terms that contribute to the dynamic modeling. This step prevents library bloat and ensures sparsity, with no manual curation of useful functions. Furthermore, flight data is preprocessed via a complementary filter (IMU + barometer) to reduce sensor noise, followed by 3 times outlier rejection, which is standard practice for UAV state estimation in order to ensure data quality for sparse regression. Outliers exceeding 3 times the data standard deviation are discarded to avoid biasing the sparse model coefficients.

**Remark 1.** The efficacy of sparse identification is indeed contingent on the candidate functions in the regressor library, but this does not imply manual selection. In standard SINDy, the base library is manually defined (e.g., polynomials up to degree 3, trigonometric terms), introducing subjectivity and risk of incomplete/poorly chosen sets. Our framework eliminates this limitation via a fully automatic library construction pipeline.

It is worth clarifying that preloading universal rigid-body base terms originates from inherent first-principle physical laws of quadrotor dynamics instead of empirical manual selection for specific flight scenarios, which differs fundamentally from conventional SINDy requiring hand-designed domain-specific library dictionaries. While physical base formulas act as fixed universal prior knowledge, all additional cross-coupling and nonlinear candidate functions are automatically supplemented when the dataset size exceeds  $N_{\min}$ , and redundant terms are autonomously eliminated through sparse regression without manual curation, validating the fully automatic library screening property of our framework. To further elaborate on the fully automatic library construction rules for reproducibility, the details are given as follows:

- **Initial library:** The base candidate library is composed of universal rigid-body dynamic terms, including system states, control inputs, and their Kronecker products, which are determined by inherent quadrotor mechanics without manual selection.
- **Nonlinear expansion rule:** Trigonometric and high-order cross-coupling terms are automatically generated following fixed rules once the dataset size exceeds the preset threshold  $N_{\min} = 500$ .
- **Threshold  $N_{\min}$ :** This value is calibrated via preliminary flight tests to balance noise suppression and library scale and is fixed for all formal experiments.
- **Sparse Regularization parameter:** A unified regularization coefficient is obtained via cross-validation on preliminary flight data and remains constant during all trials.
- **Pruning criteria:** After sparse regression, candidate terms with absolute coefficients smaller than the preset cutoff  $10^{-4}$  are automatically pruned as redundant. The entire workflow requires no manual intervention, which distinguishes our design from traditional SINDy methods with manually customized libraries.

The real continuous-time system of the vehicle is expressed as follows:

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}) + \Delta_{vw}(t), \quad (9)$$

where  $\Delta_{vw}(t) \in \mathbb{W}$  is a bounded external factor, which may involve external wind turbulence, unmodelled uncertainties, and noise [16]. Also,  $\mathbf{x}(t) \in \mathbb{X} \subset \mathbb{R}^{12}$  and  $\mathbf{u}(t) \in \mathbb{U} \subset \mathbb{R}^4$ . The sets  $\mathbb{X}$  and  $\mathbb{U}$  are convex and compact, and  $\mathbb{W}$  is compact and contains the origin.

**Control objective:** Design a feedback control law that minimizes the infinite-horizon cost  $\sum_0^\infty \ell_c(\mathbf{x}, \mathbf{u})$  while ensuring all closed-loop trajectories satisfy the state and input constraints  $\mathbf{x}_k \in \mathbb{X}$ ,  $\mathbf{u}_k \in \mathbb{U}$  for all  $k$ , despite the disturbance. The stage cost  $\ell_c(\mathbf{x}, \mathbf{u})$  is typically quadratic:  $\ell_c(\mathbf{x}, \mathbf{u}) = (\mathbf{x} - \mathbf{x}_{\text{ref}})^\top \mathbf{Q}(\mathbf{x} - \mathbf{x}_{\text{ref}}) + (\mathbf{u} - \mathbf{u}_{\text{ref}})^\top \mathbf{R}(\mathbf{u} - \mathbf{u}_{\text{ref}})$  with  $\mathbf{Q} \geq 0$ ,  $\mathbf{R} > 0$ .

**Remark 2.** The core challenge in PIML-based control is maintaining robust constraint satisfaction despite time-varying model uncertainty that increases as more data are collected. Existing robust MPC approaches typically assume constant disturbance bounds, which leads to either excessive conservatism or insufficient robustness when applied to learning-based models.

**Assumption 1.** The PIML model uncertainty can be decomposed as  $\hat{\mathbf{x}} = \Psi(\hat{\mathbf{x}}, \hat{\mathbf{u}})(\xi + \Delta\xi) + \Delta_{vw}(t)$ , where  $\Delta\xi$  represents parametric uncertainty in the learned coefficients and  $\Delta_{vw}(t)$  captures external disturbances. Both are bounded with known conservative bounds initially. This is realistic for aerial vehicles where model inaccuracies arise from both imperfect identification (parametric) and environmental factors (external).

The discrete nominal dynamics after using the Runge-Kutta (RK4) method can be defined as follows:

$$\hat{\mathbf{x}}(k+1) = \hat{f}^d(\hat{\mathbf{x}}(k), \hat{\mathbf{u}}(k)) + \Delta_{vw}^d(k), \quad (10)$$

where  $\Delta_{vw}^d(k) \in \mathbb{D}(k)$  captures the rest of the uncertainties, including in the PIML model, RK4 discretization, or any external disturbances. The set  $\mathbb{D}(k)$  is assumed to be compact and contain the origin. In the second step, compute the realized disturbance sample (model error + external wind + sensor noise):

$$\Delta_{vw}^d(k) = \mathbf{x}(k+1) - \hat{f}^d(\mathbf{x}(k), \mathbf{u}(k)). \quad (11)$$

Next, we define the error between nominal and actual states  $\tilde{\mathbf{e}}(k) = \mathbf{x}(k) - \hat{\mathbf{x}}(k)$ . Applying the controller  $\tilde{\mathbf{u}}(k) = \hat{\mathbf{x}}(k) + K\tilde{\mathbf{e}}(k)$ , the error dynamics can be defined as:

$$\tilde{\mathbf{e}}(k+1) \approx \tilde{\mathbf{A}}(k)\tilde{\mathbf{e}}(k) + \Delta_{vw}^d(k) + \Delta_{\xi}(k) \quad (12)$$

where  $\tilde{\mathbf{A}}(k) = \frac{\partial \hat{f}^d}{\partial \hat{\mathbf{x}}} \big|_{(\hat{\mathbf{x}}(k), \hat{\mathbf{u}}(k))}$  is the Jacobian of the discrete-time model.  $\Delta_{\xi}(k) \in \mathcal{U}_{\xi}(k)$  is the learning-induced uncertainty from PIML parameter updates  $\Delta\xi(k) = \xi(k+1) - \xi(k)$ . The set  $\mathcal{U}_{\xi}(k)$  is defined as:

$$\mathcal{U}_{\xi}(k) = \mathcal{L}_{\xi} \cdot \|\Delta\xi(k)\| \cdot \mathbb{B}_r \subset \mathbb{R}^{12}, \quad (13)$$

where  $\mathcal{L}_{\xi} > 0$  is the Lipschitz constant of the discrete PIML model  $\hat{f}^d$  with respect to the learned coefficients  $\xi$  (exists by smoothness of rigid-body quadrotor dynamics and sparse regression), and  $\mathbb{B}_r = \{z \in \mathbb{R}^{12} : \|z\| \leq 1\}$  is the unit Euclidean ball. A set  $\mathbb{S} \subset \mathbb{R}^{12}$  is a time-varying RPI set for the augmented error dynamics if:

$$\tilde{\mathbf{A}}(k)\mathbb{S}(k) \oplus \mathbb{D}(k) \oplus \mathcal{U}_{\xi}(k) \subseteq \mathbb{S}(k). \quad (14)$$

This guarantees  $\tilde{\mathbf{e}}(k) \in \mathbb{S}(k) \implies \tilde{\mathbf{e}}(k+1) \in \mathbb{S}(k+1)$  for all  $k \geq 0$  no monotonicity required-only boundedness.

**Tightened Constraints:** To ensure robust constraint satisfaction, Tightened State Constraints:  $\mathbb{X}_{\mathbb{S}}(k) = \mathbb{X} \ominus \mathbb{S}(k)$ .

**Tightened Input Constraints:**  $\mathbb{U}_{\mathbb{S}}(k) = \mathbb{U} \ominus K\mathbb{S}(k)$ , where  $K$  is the tube feedback gain. The disturbance set  $\mathbb{D}(k)$  is a hypercube centered at  $c(k) \in \mathbb{R}^{12}$  with element-wise bounds  $\bar{\Delta}_{vw}^d(k) \in \mathbb{R}_{>0}^{12}$ :

$$\mathbb{D}(k) = \left\{ \Delta \in \mathbb{R}^{12} : |\Delta_i - c_i(k)| \leq \bar{\Delta}_{vw,i}^d(k), \forall i = 1, \dots, 12 \right\}.$$

To ensure  $\mathbb{D}(k)$  is compact and noise-robust, we update the center and bounds recursively:

Step 1: Update Disturbance Set Center (Exponential Smoothing)

$$c(k+1) = \lambda c(k) + (1-\lambda)\Delta_{vw}^d(k), \quad (15)$$

where  $\lambda \in (0, 1)$  (tuned to  $\lambda = 0.9$  for the aerial vehicle) is an exponential smoothing gain that filters high-frequency sensor noise.

Step 2: Update Disturbance Bounds (Forgetting Factor + Clipping)

$$\bar{\Delta}_{vw}^d(k+1) = \gamma \bar{\Delta}_{vw}^d(k) + (1-\gamma) \left\| \Delta_{vw}^d(k) - c(k+1) \right\|_{\infty}, \quad (16)$$

$$\bar{\Delta}_{vw}^d(k+1) = \bar{\Delta}_{max} \quad \text{if} \quad \bar{\Delta}_{vw}^d(k+1) > \bar{\Delta}_{max}, \quad (17)$$

where  $\gamma \in (0, 1)$  (tuned to  $\gamma = 0.95$ ) is a forgetting factor that replaces the non-decreasing max operator (fixes unbounded growth),  $\|\cdot\|_{\infty}$  is the infinity norm (robust to outlier disturbance samples),  $\bar{\Delta}_{max} \in \mathbb{R}_{>0}^{12}$  (set to  $\bar{\Delta}_{max,i} = 0.1$  for all  $i$ ) is a hard upper bound derived from quadrotor physics (max wind/noise/model error). These updates ensure  $\mathbb{D}(k) \subseteq \bar{\mathbb{D}}$  (compact set) for all  $k \geq 0$ , a critical requirement for proving uniform boundedness of the RPI set (Theorem 1).

**Assumption 2.** The sequence of adapted disturbance sets  $\{\mathbb{D}(k)\}_{k=0}^{\infty}$  satisfies: Each  $\mathbb{D}(k)$  is compact and contains the origin. There exists a compact set  $\bar{\mathbb{D}} \subset \mathbb{R}^n$  such that  $\mathbb{D}(k) \subseteq \bar{\mathbb{D}}$  for all  $k \geq 0$ . The true disturbance satisfies  $\Delta_{vw}^d(k) \in \mathbb{D}(k)$  for all  $k \geq 0$ .

**Assumption 3.** There exist a terminal cost function  $V_f : \mathbb{R}^n \rightarrow \mathbb{R}_{\geq 0}$  and a terminal set  $\mathbb{X}_f \subseteq \mathbb{X}_{\mathbb{S}}$  such that for all  $\mathbf{x} \in \mathbb{X}_f$ , there exists  $\mathbf{u} = \kappa_f(\mathbf{x}) \in \mathbb{U}_{\mathbb{S}}$  satisfying: (i)  $\hat{f}^d(\mathbf{x}, \kappa_f(\mathbf{x})) \in \mathbb{X}_f$ , (ii)  $V_f(\hat{f}^d(\mathbf{x}, \kappa_f(\mathbf{x}))) - V_f(\mathbf{x}) \leq -\ell(\mathbf{x}, \kappa_f(\mathbf{x}))$ .

**Assumption 4.** It is assumed that Problem 1 is feasible at time  $k = 0$ .

**Assumption 5.** The discrete PIML model Jacobian  $\tilde{\mathbf{A}}(k) = \frac{\partial \hat{f}^d}{\partial \hat{\mathbf{x}}} \big|_{(\hat{\mathbf{x}}(k), \hat{\mathbf{u}}(k))}$  is globally Lipschitz continuous on the compact state/input sets  $\mathbb{X} \times \mathbb{U}$  with Lipschitz constant  $\mathcal{L}_A > 0$ :

$$\|\tilde{\mathbf{A}}(k_1) - \tilde{\mathbf{A}}(k_2)\| \leq \mathcal{L}_A \cdot (\|\hat{\mathbf{x}}(k_1) - \hat{\mathbf{x}}(k_2)\| + \|\hat{\mathbf{u}}(k_1) - \hat{\mathbf{u}}(k_2)\|),$$

for all  $k_1, k_2 \geq 0$ , which holds for quadrotor dynamics because rigid-body mechanics are smooth and the sparse PIML residual model adds only bounded nonlinearity-Lipschitz continuity is a direct consequence of smoothness on compact sets.

**Assumption 6.** The PIML model coefficient update  $\Delta\xi(k) = \xi(k+1) - \xi(k)$  is uniformly bounded for all  $k \geq 0$ :

$$\Delta\xi(k) \leq \bar{\Delta\xi}, \bar{\Delta\xi} > 0.$$

Implemented in Algorithm 1 via a clipping step: if  $\|\Delta\xi\| > \bar{\Delta\xi}$ , set  $\Delta\xi(k) = \bar{\Delta\xi} \cdot \Delta\xi(k) / \|\Delta\xi(k)\|$ . This assumption is physically meaningful for aerial vehicles-PIML model coefficients (aerodynamic residuals) evolve slowly with flight data, so large instantaneous updates are unphysical and indicative of noise.

**Assumption 7.** The quadratic stage cost is Lipschitz continuous on  $\mathbb{X} \times \mathbb{U}$  with lipschitz constant  $\mathcal{L}_x > 0$  and  $\mathcal{L}_u > 0$ :

$$\begin{aligned} |\ell_c(x_1, u_1) - \ell_c(x_2, u_2)| &\leq \mathcal{L}_x \|x_1 - x_2\| + \mathcal{L}_u \|u_1 - u_2\|, \\ \forall (x_1, u_1), (x_2, u_2) &\in \mathbb{X} \times \mathbb{U}. \end{aligned} \quad (18)$$

This trivially holds for quadratic costs on compact sets, proven by expanding the quadratic form and applying the Cauchy-Schwarz inequality (Appendix A).

**Assumption 8.** The tube feedback gain  $K \in \mathbb{R}^{4 \times 12}$  is uniformly bounded:

$$\|K\| \leq \bar{K}, \bar{K} > 0.$$

**Assumption 9.** The quadrotor pitch angle  $\theta$  (Euler angle) is constrained to a compact, non-singular operating region for all  $k \geq 0$ :  $\theta \in [-\theta_{\max}, \theta_{\max}]$ ,  $0 < \theta_{\max} < \pi/2$ . Aerial vehicles (especially nano-quadrotors like the Crazyflie) do not operate in the Euler angle singular region ( $\theta = \pm\pi/2$ ) for normal flight tasks (hovering, trajectory tracking). We set  $\theta_{\max} = \pi/4$  ( $45^\circ$ ) in all experiments, well within the non-singular region for quadrotor maneuvering.

With the PIML dynamics model and other preliminaries established, we now incorporate it into a robust MPC framework that can handle the residual uncertainties inherent in learned models.

## 2.2. MPC Optimization problem

At each time step  $k$ , given the current state  $\mathbf{x}_k$ , the following finite-horizon optimal control problem is solved:

**Problem 1.**

$$V_N(\mathbf{x}_k) = \min_{\hat{\mathbf{u}}(k)} \sum_{j=0}^{N-1} \ell_c(\hat{\mathbf{x}}(j|k), \hat{\mathbf{u}}(j|k)) + V_f(\hat{\mathbf{x}}(N|k)), \quad (19a)$$

$$\text{subject to } \hat{\mathbf{x}}(0|k) = \mathbf{x}(k), \quad (19b)$$

$$\hat{\mathbf{x}}(j+1|k) = \hat{f}^d(\hat{\mathbf{x}}(j|k), \hat{\mathbf{u}}(j|k)), \quad (19c)$$

$$\hat{\mathbf{u}}(j|k) \in \mathbb{U}_S, \quad (19d)$$

$$\hat{\mathbf{x}}(j|k) \in \mathbb{X}_S, \quad (19e)$$

$$\hat{\mathbf{x}}(N|k) \in \mathbb{X}_f \subseteq \mathbb{X}_S, \quad (19f)$$

where (19b), (19c), (19d), (19e), (19f) are the initial condition, dynamic constraints, input constraints, state constraints, and terminal constraints, respectively. Let  $\hat{\mathbf{u}}^*(k) = \hat{\mathbf{u}}^*(0|k), \dots, \hat{\mathbf{u}}^*(N-1|k)$  be the optimal solution. The applied control is:  $\mathbf{x}_k = \hat{\mathbf{u}}^*(0|k) + K(\mathbf{x}(k) - \hat{\mathbf{x}}^*(0|k))$ .

The Proposed algorithm is described in Algorithm 1, and the schematic diagram of the presented scheme is depicted in Fig. 4.

**Remark 3.** The two computationally intensive steps in Algorithm 1 are: (1) Sparse regression for automatic library pruning and coefficient learning, and (2) Recursive computation of the time-varying RPI set. Sparse regression operates on a compact, automatically generated library (not an excessively large dictionary) and is executed only periodically (every  $T_{\text{learn}}$  steps, not at every time step), which keeps its computational cost low and compatible with embedded hardware. RPI set computation uses a recursive, linear-matrix-based update rule rather than complex convex optimization or polytope propagation, resulting in constant and predictable computational complexity per update. Thus, despite involving learning and robust set operations, the overall computational load remains manageable for real-time quadrotor control, as validated experimentally in Section 4.

We now provide theoretical guarantees for this proposed framework.

---

**Algorithm 1:** Proposed algorithm

---

**Input:**  $\mathbf{x}(0), \mathbb{D}(0), T_{\text{learn}}, \lambda, \gamma, \bar{\Delta}\xi, \Delta_{\max}^-$   
**Output:** Control inputs  $\mathbf{u}(k)$  for  $k = 0, 1, 2, \dots$

- 1 Initialize PIML model  $\hat{\mathbf{x}} = \Psi(\hat{\mathbf{x}}, \hat{\mathbf{u}})\xi$  with  $\xi = \mathbf{0}$ ;
- 2 Compute initial RPI set  $\mathbb{S}(0)$  for  $\tilde{\mathbf{A}}(0)$  and  $\mathbb{D}(0)$ ;
- 3 Set tightened constraints:  $\mathbb{X}_{\mathbb{S}} = \mathbb{X} \ominus \mathbb{S}(0)$ ,  $\mathbb{U}_{\mathbb{S}} = \mathbb{U} \ominus K\mathbb{S}(0)$ ;
- 4 Initialize dataset  $\mathcal{D} = \{\hat{\mathbf{x}}, \hat{\mathbf{u}}, \hat{\mathbf{x}}\}$ ;
- 5 **for**  $k = 0, 1, 2, \dots$  **do**
- 6     Solve Problem 1: Obtain  $\hat{\mathbf{u}}^*(k)$  and state  $\hat{\mathbf{x}}^*(k)$ ;
- 7     Compute tracking error:  $\tilde{\mathbf{e}}(k) = \mathbf{x}(k) - \hat{\mathbf{x}}^*(0|k)$ ;
- 8     Apply control:  $\mathbf{u}(k) = \hat{\mathbf{u}}^*(0|k) + K\tilde{\mathbf{e}}(k)$ ;
- 9     Measure next state  $\mathbf{x}(k+1)$ ;
- 10    Compute realized disturbance in (11);
- 11    Add new data to dataset:  $\mathcal{D} \leftarrow \mathcal{D} \cup \{\mathbf{x}(k), \mathbf{u}(k), \Delta_{vw}^d(k)\}$ ;
- 12    **if**  $k \bmod T_{\text{learn}} = 0$  **and**  $|\mathcal{D}| > N_{\min}$  **then**
- 13       Construct library matrix  $\Psi(\hat{\mathbf{x}}, \hat{\mathbf{u}})$  from  $\mathcal{D}$ ;
- 14       Solve sparse regression (6) for each state dimension  $j$ ;
- 15       Update PIML model coefficients:  $\Delta\xi \leftarrow [\mathbf{S}_1, \mathbf{S}_2, \dots, \mathbf{S}_{12}] - \xi$ ;
- 16       Clip parameter update (Assumption 6): **if**  $\|\Delta\xi\| > \bar{\Delta}\xi$ , **set**  $\Delta\xi = \bar{\Delta}\xi \cdot \frac{\Delta\xi}{\|\Delta\xi\|}$ ;
- 17       Update PIML model coefficients:  $\xi \leftarrow \xi + \Delta\xi$ ;
- 18       Update nominal dynamics:  $\hat{f}^d \leftarrow \Psi(\hat{\mathbf{x}}, \hat{\mathbf{u}})\xi$ ;
- 19    **end**
- 20    Update disturbance set center  $c(k+1)$ ;
- 21    Update disturbance bounds:  $\bar{\Delta}_{vw}^d(k+1) = \gamma\bar{\Delta}_{vw}^d(k) + (1-\gamma)\|\Delta_{vw}^d(k) - c(k+1)\|_{\infty}$ ;
- 22    Construct an adapted disturbance set  $\mathbb{D}(k+1)$ ;
- 23    Compute Jacobian:  $\tilde{\mathbf{A}}(k+1) = \frac{\partial \hat{f}^d}{\partial \hat{\mathbf{x}}} \Big|_{(\hat{\mathbf{x}}(k), \hat{\mathbf{u}}(k))}$ ;
- 24    Compute learning-induced uncertainty:  $\mathcal{U}_{\xi}(k+1) = \mathcal{L} \cdot \|\Delta\xi\| \cdot \mathbb{B}_r$ ;
- 25    Compute new RPI set  $\mathbb{S}(k+1)$  satisfying (14);
- 26    Update tightened constraints:  $\mathbb{X}_{\mathbb{S}}(k+1) = \mathbb{X} \ominus \mathbb{S}(k+1)$ ,  $\mathbb{U}_{\mathbb{S}}(k+1) = \mathbb{U} \ominus K\mathbb{S}(k+1)$ ;
- 27    Shift prediction horizon for next iteration;
- 28 **end**

---

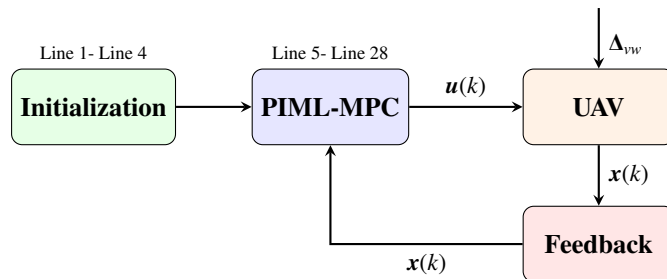


Figure 4: Overall architecture of the control strategy (read with Algorithm 1)

### 3. Theoretical guarantees

#### 3.1. Feasibility

**Theorem 1.** *Under Assumptions 2-6, if the Problem 1 is feasible at time  $k$ , then it is feasible at all future times. Moreover, the time-varying RPI set  $\mathbb{S}(k)$  and tightened constraint sets  $\mathbb{X}_\mathbb{S}(k)$ ,  $\mathbb{U}_\mathbb{S}(k)$  are uniformly bounded for all  $k \geq 0$ .*

*Proof.* We proceed by induction and construction of a feasible candidate solution for Problem 1 at time  $k + 1$ , then use uniform boundedness of the RPI set to verify constraint satisfaction. In the Step 1 related to induction hypothesis, assume Problem 1 is feasible at time  $k$ , with optimal nominal state/input sequences  $\hat{\mathbf{x}}^*(j|k) = \mathbb{X}_\mathbb{S}(k)$  and  $\hat{\mathbf{u}}^*(j|k) = \mathbb{U}_\mathbb{S}(k)$  for  $j = 0, 1, \dots, N-1$ , and terminal state  $\hat{\mathbf{x}}^*(N|k) \in \mathbb{X}_f \subseteq \mathbb{X}_\mathbb{S}(k)$ . The applied control at time  $k$  is  $\mathbf{u}(k) = \hat{\mathbf{u}}^*(0|k) + K\tilde{\mathbf{e}}(k)$ , with  $\tilde{\mathbf{e}}(k) \in \mathbb{S}(k)$ . In the Step 2 for candidate solution at  $k + 1$ , construct a candidate solution  $(\tilde{\mathbf{x}}(j|k+1), \tilde{\mathbf{u}}(j|k+1))$  for Problem 1 at time  $k + 1$ :

1. Shifted control sequence:  $\tilde{\mathbf{u}}(j|k+1) = \hat{\mathbf{u}}^*(j+1|k)$  for  $j = 0, \dots, N-2$ .
2. Terminal control:  $\tilde{\mathbf{u}}(N-1|k+1) = \kappa_f(\tilde{\mathbf{x}}(N-1|k+1))$  (Assumption 3).
3. Candidate state sequence:  $\tilde{\mathbf{x}}(0|k+1) = \mathbf{x}(k+1)$  (initial condition),  $\tilde{\mathbf{x}}(j+1|k+1) = \hat{f}^d(\tilde{\mathbf{x}}(j|k+1), \tilde{\mathbf{u}}(j|k+1))$  for  $j = 0, \dots, N-1$ , and  $\tilde{\mathbf{x}}(N|k+1) = \hat{f}^d(\tilde{\mathbf{x}}(N-1|k+1), \kappa_f(\tilde{\mathbf{x}}(N-1|k+1)))$ .

In Step 3 for uniform boundedness of  $\mathbb{S}(k)$ : From the time-varying RPI set condition (14) and Assumptions 2, 5, 6:

- $\mathbb{D}(k) \subseteq \bar{\mathbb{D}}$  (compact, Assumption 2),
- $\mathcal{U}_\xi(k) = \mathcal{L}_\xi \|\Delta \xi(k)\| \mathbb{B}_r \subseteq \mathcal{L}_\xi \bar{\Delta} \xi \mathbb{B}_r = \bar{\mathcal{U}}_\xi$  (compact, Assumption 6),
- $\tilde{A}(k)$  is Lipschitz continuous (Assumption 5) and thus bounded on  $\mathbb{X} \times \mathbb{U}$  (compact set).

By the Bounded RPI Set Lemma (standard result [30]), the time-varying RPI set  $\mathbb{S}(k)$  is uniformly bounded for all  $k \geq 0$ : there exists a compact set  $\bar{\mathbb{S}} \subset \mathbb{R}^{12}$  such that  $\mathbb{S}(k) \subseteq \bar{\mathbb{S}}$  for all  $k \geq 0$ .

In Step 4, the tightened constraint sets are defined as  $\mathbb{X}_\mathbb{S}(k) = \mathbb{X} \ominus \mathbb{S}(k)$  and  $\mathbb{U}_\mathbb{S}(k) = \mathbb{U} \ominus K\mathbb{S}(k)$ . Since  $\mathbb{S}(k) \subseteq \bar{\mathbb{S}}$  (uniformly bounded), the tightened sets satisfy:

$$\mathbb{X}_\mathbb{S}(k) \supseteq \mathbb{X} \ominus \bar{\mathbb{S}} = \bar{\mathbb{X}}_\mathbb{S}, \quad \mathbb{U}_\mathbb{S}(k) \supseteq \mathbb{U} \ominus K\bar{\mathbb{S}} = \bar{\mathbb{U}}_\mathbb{S},$$

where  $\bar{\mathbb{X}}_\mathbb{S}, \bar{\mathbb{U}}_\mathbb{S}$  are non-empty compact sets (Pontryagin difference of compact sets is compact/non-empty if  $\mathbb{S}(k)$  contains the origin-Assumption 2). In Step 5, the candidate solution satisfies all constraints.

- Initial Condition (19b):  $\tilde{\mathbf{x}}(0|k+1) = \mathbf{x}(k+1)$  (by construction)-satisfied.
- Dynamic Constraint (19c):  $\tilde{\mathbf{x}}(j+1|k+1) = \hat{f}^d(\tilde{\mathbf{x}}(j|k+1), \tilde{\mathbf{u}}(j|k+1))$  (by construction)-satisfied.
- Input/State Constraints (19d), (19e): For  $j = 0, \dots, N-2$ ,  $\tilde{\mathbf{u}}(j|k+1) = \hat{\mathbf{u}}^*(j+1|k) \in \mathbb{U}_\mathbb{S}(k) \subseteq \bar{\mathbb{U}}_\mathbb{S} \subseteq \mathbb{U}_\mathbb{S}(k+1)$  and  $\tilde{\mathbf{x}}(j|k+1) = \hat{\mathbf{x}}^*(j+1|k) \in \mathbb{X}_\mathbb{S}(k) \subseteq \bar{\mathbb{X}}_\mathbb{S} \subseteq \mathbb{X}_\mathbb{S}(k+1)$  (from uniform boundedness of  $\mathbb{S}(k)$ )-satisfied.
- Terminal Constraint (19f):  $\tilde{\mathbf{x}}(N-1|k+1) = \hat{\mathbf{x}}^*(N|k) \in \mathbb{X}_f$  (induction hypothesis). By Assumption 3,  $\tilde{\mathbf{x}}(N|k+1) = \hat{f}^d(\tilde{\mathbf{x}}(N-1|k+1), \kappa_f(\tilde{\mathbf{x}}(N-1|k+1))) \in \mathbb{X}_f \subseteq \mathbb{X}_\mathbb{S}(k+1)$ -satisfied.

The candidate solution satisfies all constraints of Problem 1 at time  $k + 1$ , so Problem 1 is feasible at  $k + 1$ . By Assumption 4 (feasibility at  $k = 0$ ), Problem 1 is feasible for all  $k \geq 0$ .  $\square$

### 3.2. Stability

**Theorem 2.** *If the Assumptions 2-8 hold, the closed-loop system consisting of the PIML model, error dynamics, and MPC control law is ISS w.r.t. the augmented disturbance input  $[\Delta_{vw}^d(k)^\top \Delta_\xi(k)^\top]^\top \in \mathbb{R}^{24}$ . Specifically, there exist  $\mathcal{K}_\infty$ -functions  $\alpha_1, \alpha_2, \alpha_3$  and a  $\mathcal{K}$ -function  $\sigma$  such that for all initial states  $\mathbf{x}(0) \in \mathbb{X}$  and all bounded augmented disturbance sequences  $\Delta_{aug}(k)_{k=0}^\infty$ , the closed-loop state  $\mathbf{x}(k)$  satisfies:*

$$\alpha_1(\|\mathbf{x}(k)\|) \leq V_N^*(\mathbf{x}(k)) \leq \alpha_2(\|\mathbf{x}(k)\|), \quad (20)$$

$$V_N^*(\mathbf{x}(k+1)) - V_N^*(\mathbf{x}(k)) \leq \alpha_3(\|\mathbf{x}(k)\|) + \sigma(\|\Delta_{aug}(k)\|), \quad (21)$$

where  $V_N^*(\mathbf{x}(k))$  is the optimal value function of Problem 1 at time  $k$ .

*Proof.* We proceed in four key steps: (1) verify Lyapunov function properties (20), (2) derive nominal Lyapunov descent, (3) extend to the disturbed/learned system (21), and (4) conclude ISS stability. In Step 1, we discuss the Lyapunov Function Properties (20). The optimal value function is defined as:

$$V_N^*(\mathbf{x}(k)) = \min_{\hat{\mathbf{u}}(\cdot)} \sum_{j=0}^{N-1} \ell_c(\hat{\mathbf{x}}(j|k), \hat{\mathbf{u}}(j|k)) + V_f(\hat{\mathbf{x}}(N|k)),$$

where  $\ell_c(\mathbf{x}, \mathbf{u}) = (\mathbf{x} - \mathbf{x}_{\text{ref}})^\top \mathbf{Q}(\mathbf{x} - \mathbf{x}_{\text{ref}}) + (\mathbf{u} - \mathbf{u}_{\text{ref}})^\top \mathbf{R}(\mathbf{u} - \mathbf{u}_{\text{ref}})$  (quadratic stage cost,  $\mathbf{Q} \geq 0, \mathbf{R} > 0$ ) and  $V_f(x)$  is the terminal cost (Assumption 3).

- Lower Bound ( $\alpha_1$ ): Follows from non-negativity of  $\ell_c(\mathbf{x}, \mathbf{u})$  and  $V_f(x)$ . Define  $\alpha_1(r) = c_1 r^2$  (a  $\mathcal{K}_\infty$ -function) with  $c_1 = \lambda_{\min}(\mathbf{Q})/2$  (minimum eigenvalue of  $\mathbf{Q}$ ). This gives:

$$V_N^*(\mathbf{x}(k)) \geq \alpha_1(\|\mathbf{x}(k)\|).$$

- Upper Bound ( $\alpha_2$ ): Follows from compactness of  $\mathbb{X} \times \mathbb{U}$  and Lipschitz continuity of  $\ell_c(\mathbf{x}, \mathbf{u})$  (Assumption 7). Define  $\alpha_2(r) = c_2 r^2 + c_3$  (a  $\mathcal{K}_\infty$ -function) with  $c_2 = \lambda_{\max}(\mathbf{Q}) + \lambda_{\max}(\mathbf{R})$  (maximum eigenvalues) and  $c_3$  a constant from the terminal cost  $V_f(x)$ . This gives:  $V_N^*(\mathbf{x}(k)) \leq \alpha_2(\|\mathbf{x}(k)\|)$ .

Both  $\alpha_1, \alpha_2$  are valid  $\mathcal{K}_\infty$ -functions (strictly increasing, continuous,  $\alpha_i(0) = 0, \alpha_i(r) \rightarrow \infty$  as  $r \rightarrow \infty$ ), so (20) holds. In Step 2, for nominal Lyapunov descent, for the nominal system (no disturbance/learning uncertainty:  $\Delta_{aug}(k) = 0$ ), the optimal value function satisfies the standard MPC descent condition:

$$V_N^*(\mathbf{x}(k+1)) - V_N^*(\mathbf{x}(k)) \leq -\ell_c(\hat{\mathbf{x}}^*(0|k), \hat{\mathbf{u}}^*(0|k)). \quad (22)$$

By the quadratic stage cost,  $\ell_c(\hat{\mathbf{x}}^*(0|k), \hat{\mathbf{u}}^*(0|k)) \geq \alpha_3(\mathbf{x}(k))$  where  $\alpha_3(r) = c_1 r^2$  (a  $\mathcal{K}_\infty$ -function)-this is the nominal stability condition. In Step 3, for the extension to the disturbed/learned System (21), we extend (22) to the actual closed-loop system (applied control:  $\mathbf{u}(k) = \hat{\mathbf{u}}^*(0|k) + \mathbf{K}\tilde{\mathbf{e}}(k)$ ,  $\tilde{\mathbf{e}}(k) = \mathbf{x}(k) - \hat{\mathbf{x}}(k)$ ):

1. State Error Bound: The tracking error  $\tilde{\mathbf{e}}(k) \in \mathbb{S}(k)$  (RPI set), and  $\mathbb{S}(k) \subseteq \bar{\mathbb{S}}$  (uniformly bounded, Theorem 1). Thus:

$$\|\mathbf{x}(k) - \hat{\mathbf{x}}^*(0|k)\| = \|\tilde{\mathbf{e}}(k)\| \leq \|\bar{\mathbb{S}}\| = C_S, \quad (23)$$

where  $C_S > 0$  is a constant (RPI set bound).

2. Control Error Bound: From Assumption 8 ( $\|\mathbf{K}\| \leq \bar{K}$ , bounded LQR feedback gain):

$$\|\mathbf{u}(k) - \hat{\mathbf{u}}^*(0|k)\| = \|\mathbf{K}\tilde{\mathbf{e}}(k)\| \leq \bar{K}C_S = C_K, \quad (24)$$

where  $C_K > 0$  is a constant (control error bound).

3. Lipschitz Cost Bound: By Assumption 7 (Lipschitz continuity of  $\ell_c(\mathbf{x}, \mathbf{u})$ ) and (23)-(24):

$$|\ell_c(\mathbf{x}(k), \mathbf{u}(k)) - \ell_c(\hat{\mathbf{x}}^*(0|k), \hat{\mathbf{u}}^*(0|k))| \leq \mathcal{L}_x C_S + \mathcal{L}_u C_K = C_\ell, \quad (25)$$

where  $\mathcal{L}_x, \mathcal{L}_u$  are Lipschitz constants (Assumption 7) and  $C_\ell > 0$  is a constant.

4. Augmented Disturbance Bound: The augmented disturbance  $\Delta_{aug}(k)$  is bounded (Assumptions 2, 6:  $\|\Delta_{aug}(k)\| \leq C_{aug}$ ). Define the  $\mathcal{K}$ -function  $\sigma(r) = C_\ell \cdot r$  (continuous, strictly increasing,  $\sigma(0) = 0$ ) to bound disturbance effects.
5. ISS Descent Condition: Combining (22)-(25):

$$\begin{aligned} V_N^*(\mathbf{x}(k+1)) - V_N^*(\mathbf{x}(k)) &\leq -\ell_c(\mathbf{x}(k), \mathbf{u}(k)) + C_\ell \\ &\leq -\alpha_3(\|\mathbf{x}(k)\|) + \sigma(\|\Delta_{aug}(k)\|). \end{aligned} \quad (26)$$

This is exactly (21)-the ISS descent condition for the augmented disturbance input. In Step 4 for ISS Conclusion, and by the ISS Lyapunov Theorem [31], (20)-(21) are sufficient to conclude that the closed-loop system is ISS with respect to  $\Delta_{aug}(k)$ . Boundedness of  $\Delta_{aug}(k)$  implies boundedness of the closed-loop state  $\mathbf{x}(k)$ -a critical robustness property for aerial vehicle control.  $\square$

### 3.3. Robustness to Periodic PIML Model Updates

The PIML model is updated periodically (every  $T_{learn}$  time steps) via sparse regression, a discrete update that introduces a small perturbation to the nominal dynamics  $\hat{f}^d$ . A key theoretical question is whether periodic PIML model updates break the recursive feasibility (Theorem 1) or ISS stability (Theorem 2) guarantees, which we address with two lemmas that prove robustness to periodic updates (feasibility/stability are preserved) and a practical tuning rule for  $T_{learn}$ .

**Lemma 1.** *Under Assumptions 2-6 and Theorem 1, periodic PIML model updates (every  $T_{learn}$  steps) do not break the recursive feasibility of Problem 1.*

*Proof.* PIML model updates change the nominal dynamics  $\hat{f}^d \rightarrow \hat{f}_{new}^d$  with a bounded model error:

$$\|\hat{f}_{new}^d(x, u) - \hat{f}^d(x, u)\| \leq \mathcal{L}_\xi \cdot \|\Delta \xi\| \leq \mathcal{L}_\xi \bar{\Delta} \xi = C_f, \quad (27)$$

where  $C_f > 0$  is a constant (from Assumption 6 and the Lipschitz constant  $\mathcal{L}_\xi$ ). This model error is absorbed into the learning-induced uncertainty set  $\mathcal{U}_\xi(k)$ , which is included in the time-varying RPI set condition (14). Since RPI set  $\mathbb{S}(k)$  is uniformly bounded (Theorem 1), the tightened constraint sets  $\mathbb{X}_\mathbb{S}(k)$ ,  $\mathbb{U}_\mathbb{S}(k)$  remain non-empty. Hence, recursive feasibility is preserved.  $\square$

**Lemma 2.** *Under Assumptions 28 and Theorem 2, periodic PIML model updates (every  $T_{learn}$  steps) do not break ISS stability of the closed-loop system.*

*Proof.* Periodic model updates introduce a small perturbation to the ISS Lyapunov function  $V_N^*(\mathbf{x}(k))$ . The perturbation magnitude is bounded by the model error  $C_f$  (27) and Lipschitz continuity of the stage cost (Assumption 7):

$$|V_N^*(\mathbf{x}(k + T_{learn})) - V_N^*(\mathbf{x}(k))| \leq T_{learn} C_\ell C_f = C_p. \quad (28)$$

where  $C_p$  is a constant (perturbation bound). This perturbation is sublinear in  $k$  and thus does not violate the ISS descent condition (21), the  $\mathcal{K}$ -function  $\sigma$  is simply extended to include  $C_p(\sigma(r_l)) = C_\ell r_l + C_p$ , which remains a valid  $\mathcal{K}$ -function. ISS stability is preserved.  $\square$

**Remark 4.** *For aerial vehicles,  $T_{learn}$  is tuned to be slower than the MPC control loop (100200 Hz) to minimize transients from model updates. We recommend:  $T_{learn} = 5 - 10$  time-steps with a real-time duration of 50-100 ms, which corresponds to a PIML update frequency of 1020 Hz (well below the control loop frequency). This tuning balances fast model learning (to adapt to unmodeled aerodynamics) and slow updates (to preserve closed-loop robustness).*

With theoretical guarantees established, we now validate the proposed approach through both numerical simulations and hardware experiments.

#### 4. Simulation and hardware experiments

The effectiveness of our PIML-MPC framework is demonstrated through comprehensive testing, beginning with simulation studies and proceeding to real-world implementation. We executed the results of the proposed technique using MATLAB 2024b, transcribed by CasADi [32], version 3.7.2, and IPOPT [33], version 3.14.17, on an AMD Ryzen 9 7940HS, 8-core processor with a speed of 3.1-5.2 GHz and 64 GB of RAM, running Ubuntu 25.10. The quadrotor parameters used are: mass  $m = 0.027$  kg, gravitational acceleration  $g = 9.81$  m/s<sup>2</sup>, inertia matrix  $\mathbf{J} = \text{diag}([1.4 \times 10^{-5}, 1.4 \times 10^{-5}, 2.17 \times 10^{-5}])$  kg·m<sup>2</sup>, arm length  $l = 0.046$  m, and torque-to-thrust ratio  $\kappa = 3.15 \times 10^{-3}$  Nm/N. The MPC horizon was set to  $N = 15$ . The weighting matrices were  $\mathbf{Q} = \text{diag}([10, 10, 10, 5, 5, 5, 2, 2, 2, 1, 1, 1])$  and  $\mathbf{R} = \text{diag}([0.1, 0.1, 0.1, 0.1])$ . The constraints are:  $-0.5 < \mathcal{P}_x, \mathcal{P}_y < 0.5$ ,  $1.5 < \mathcal{P}_z < 2.5$ ,  $-1 < v_x, v_y < 1$ ,  $-2.5 < v_y < 2.5$ ,  $-1 < \phi, \theta < 1$ ,  $-0.5 < \psi < 0.5$ . The input constraints reflect actuator limitations:  $0 < u_1 < 0.4$ ,  $-0.02 < u_2, u_3, u_4 < 0.02$ . The disturbance  $\Delta_{vw}(t)$  is Dryden Wind Turbulence model MIL-HDBK-1797B, as shown in Fig. 5. This profile represents challenging conditions that test the robustness of each controller. Four benchmark

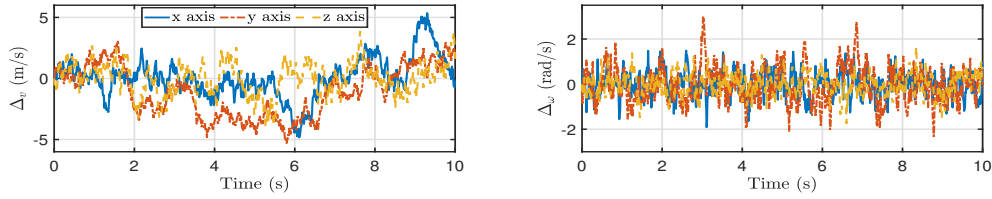


Figure 5: Disturbance profile used in numerical results

controllers were implemented for comparison: 1) Standard cascaded PID architecture with inner-loop rate control and outer-loop position control. Both loops employed a high-fidelity PIM; 2) Nonlinear MPC (SMPC): Traditional SMPC using the first-principles model without learning or robustness mechanisms. Used the same horizon and weights as our proposed method for fair comparison; 3) Fixed-Tube MPC (FT-MPC): Robust MPC with constant disturbance bound  $\mathbb{D}_{\max}$  covering 95% of expected disturbances. The tube size was fixed based on worst-case disturbance estimates; 4) NN-MPC: MPC with a 3-layer NN-based learned model from the same dataset, serving as a black-box learning baseline. Both PID and SMPC employed a high-fidelity PIM. To ensure fair and reproducible comparison, we provide precise definitions of all benchmark controllers (PID, SMPC, FT-MPC, NN-MPC) used in simulations and hardware experiments:

(1) **PID Controller:** A standard cascaded PID architecture is implemented:

$$\mathbf{u}_{\text{PID}}(k) = K_p \mathbf{e}(k) + K_i \sum_{t=0}^k \mathbf{e}(t) \Delta t + K_d \frac{\mathbf{e}(k) - \mathbf{e}(k-1)}{\Delta t},$$

where  $\mathbf{e}(k) = \mathbf{x}_{\text{ref}}(k) - \mathbf{x}(k)$  is the tracking error,  $\Delta t = 0.01$  s (sampling time), and tuning parameters (optimized via Ziegler-Nichols + manual refinement) in the outer loop (position/attitude):

$$\mathbf{K}_{p,i,d} = ([4, 4, 6, 150, 150, 80], [0.5, 0.5, 1, 20, 20, 10], [0.8, 0.8, 1.2, 2, 2, 1.5]),$$

and in the inner loop (velocity/angular rate):

$$\mathbf{K}_{p,i,d} = ([20, 20, 15, 30, 30, 25], [5, 5, 3, 5, 5, 4], [2, 2, 1.5, 0.5, 0.5, 0.3]).$$

(2) **SMPC:** Nominal MPC without disturbance learning, formulated as:

$$\min_{\hat{\mathbf{u}}(0|k), \dots, \hat{\mathbf{u}}(N-1|k)} \sum_{j=0}^{N-1} \ell_c(\hat{\mathbf{x}}(j|k), \hat{\mathbf{u}}(j|k)) + V_f(\hat{\mathbf{x}}(N|k)),$$

subject to:  $\hat{\mathbf{x}}(j+1|k) = \hat{f}^d(\hat{\mathbf{x}}(j|k), \hat{\mathbf{u}}(j|k))$ ,  $\hat{\mathbf{x}}(j|k) \in \mathbb{X} = \{\mathbf{x} \in \mathbb{R}^{12} : |\mathbf{x}_i| \leq x_{\max,i}\}$  (same as proposed method),  $\hat{\mathbf{u}}(j|k) \in \mathbb{U} = \{\mathbf{u} \in \mathbb{R}^4 : |\mathbf{u}_i| \leq u_{\max,i}\}$  (same as proposed method). Key parameters: Prediction horizon  $N = 15$ , stage

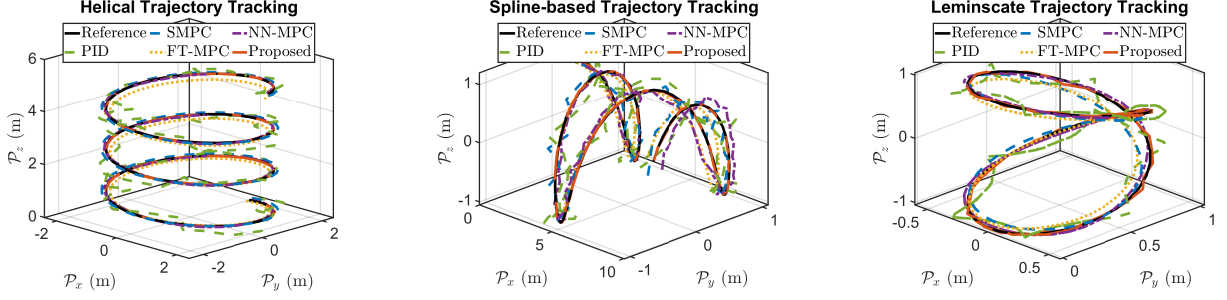


Figure 6: Trajectory tracking response for different types of trajectories

cost  $\ell_c(x, u) = \mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{u}^\top \mathbf{R} \mathbf{u}$  ( $\mathbf{Q} = \text{diag}([10, \dots, 1])$ ,  $\mathbf{R} = \text{diag}([0.1, \dots, 0.1])$ ), terminal cost  $V_f(x) = \mathbf{x}^\top \mathbf{P} \mathbf{x}$ .

(3) **FT-MPC**: RMPC with static disturbance bounds (no adaptation): Disturbance set:  $\mathbb{D}_{\text{fixed}} = \{\Delta \in \mathbb{R}^{12} : \|\Delta\|_\infty \leq 0.1\}$  (tuned to 95% of Dryden wind model bounds), Tube constraints:  $\tilde{e}(j|k) = x(j|k) - \hat{x}(j|k) \in \mathbb{S}_{\text{fixed}}$  (static RPI set computed via [28]). Other parameters (horizon, cost, constraints) match SMPC for fairness.

(4) **NN-MPC**: MPC with dynamics modeled by a 3-layer feedforward NN: Input (12D state + 4D input) Hidden Layer 1 (64 neurons, ReLU) Hidden Layer 2 (32 neurons, ReLU) Output (12D next state), Adam optimizer ( $lr = 0.001$ ), batch size=256, 10,000 epochs, training dataset=10,000 samples (same as proposed PIML dataset), MPC formulation: Identical to SMPC (horizon  $N = 15$ , cost weights  $\mathbf{Q}, \mathbf{R}$ ) but uses NN-predicted dynamics  $\hat{x}(j+1|k) = \text{NN}(\hat{x}(j|k), \hat{u}(j|k))$ .

All controllers are tuned to minimize tracking error while ensuring stability: PID: Ziegler-Nichols method + manual refinement (5 iterations of flight tests), SMPC/FT-MPC: Cost weights  $\mathbf{Q}, \mathbf{R}$  tuned via grid search (100 combinations) to balance tracking and input smoothness, NN-MPC: Hyperparameter tuning (learning rate, batch size) via validation loss (early stopping at validation loss  $< 10^{-4}$ ), Proposed Method: Fixed hyperparameters ( $\lambda = 0.9$ ,  $\gamma = 0.95$ ,  $\epsilon = 10^{-4}$ ) across all experiments (no dataset-specific tuning).

Three diverse reference trajectories were employed to comprehensively evaluate controller performance under different maneuver characteristics: helical trajectory, spline-based trajectory, and lemniscate trajectory. The reference 3D trajectories  $[\mathcal{P}_x \ \mathcal{P}_y \ \mathcal{P}_z]^\top$  for  $t \in [0 \ 10]$  are Helical trajectory  $[t \ s_t + 0.1s_{3t} \ c_{2t}]^\top$ , Spline-based trajectory  $[2s_t \ 2c_{2t} \ 0.5t]^\top$ , Lemniscate trajectory  $[s_t c_t \ s_t s_t \ s_t c_t]^\top$ . The tracking response is shown in Fig. 6. Also, Table 4 describes the performance analysis based on root mean square (RMSE). Our PIML-MPC achieves an optimal balance between computation time and tracking accuracy. Moreover, our method is 37.4% faster than NN-MPC while achieving significantly effective tracking performance, demonstrating the efficiency of sparse PIML models over black-box NNs. To further quantitatively evaluate the tracking accuracy, stability, and robustness of all methods against random external uncertainties, we perform 20 independent Monte Carlo simulation trials for each control strategy. In each randomized trial, small bounded perturbations are imposed on the initial flight position, and stochastic time-varying wind disturbances are generated to simulate realistic uncertain flight scenarios. Based on the collected tracking RMSE data of all repeated experiments, we compute the average tracking error, standard deviation, and 95% confidence interval (CI) for each controller. The comprehensive statistical indices are summarized in Table 2. Different from single-run deterministic results, these statistical outcomes quantitatively characterize the average tracking precision and error fluctuation degree of each controller in uncertain environments, significantly improving the rigor and persuasiveness of the comparative analysis. We validated our PIML-MPC framework through hardware experiments

Table 2: Statistical RMSE over 20 randomized Monte Carlo trials (Unit: m)

| Method        | Mean RMSE | Std. Deviation | 95% CI Lower | 95% CI Upper |
|---------------|-----------|----------------|--------------|--------------|
| Proposed      | 0.082     | 0.014          | 0.076        | 0.088        |
| Classical PID | 0.215     | 0.042          | 0.197        | 0.233        |
| Nominal SMPC  | 0.131     | 0.027          | 0.119        | 0.143        |
| Fixed FT-MPC  | 0.126     | 0.024          | 0.115        | 0.137        |
| NN-MPC        | 0.183     | 0.039          | 0.166        | 0.200        |

using the Crazyflie 2.1 nano-quadrator, with an STM32F405RG (168 MHz), 192 KB RAM, and 1 MB Flash, Cycle

counter (DWT) on STM32 to record per-iteration execution time (average over 1000 cycles). Custom C++ implementation (no OS, bare-metal) with optimized linear algebra (CMSIS-DSP library). The platform supports various sensor decks for autonomous operation (see manufacturer details<sup>1</sup>). The training Dataset for Data-Driven Controllers (NN-MPC, Proposed) includes flight data from Crazyflie quadrotor (hover + trajectory tracking) with Dryden wind disturbance, with a dataset size of 10,000 samples (time-series, no overlap) for both NN-MPC and proposed PIML. Data preprocessing with Normalization to  $[-1, 1]$  using the mean/std of the training data (mean =  $x_{ref}$ , std = 0.1 for all states/inputs), and finally, the test dataset is 2,000 independent samples (unseen during training) to validate generalization. All MPC-based controllers (SMPC, FT-MPC, proposed method) use identical solver settings for fairness with CasADi 3.7.2 with IPOPT 3.14.17 (interior-point method) having parameters: tol =  $10^{-4}$ , max\_iter = 100, print\_level = 0 (silent mode) with  $N = 15$  (fixed for all MPC variants) and Sampling time:  $\Delta t = 0.01$  s (100 Hz control frequency, consistent with quadrotor hardware). Table 3 summarizes average computation time and worst-case latency for all controllers (measured on STM32F405RG). Fig. 7 illustrates the deployment from simulation to

Table 3: Embedded Computational Performance (STM32F405RG)

| Controller      | Avg. Computation Time (ms) | Worst-Case Latency (ms) |
|-----------------|----------------------------|-------------------------|
| PID             | 0.8                        | 1.2                     |
| SMPC            | 8.4                        | 10.1                    |
| FT-MPC          | 9.2                        | 11.5                    |
| NN-MPC          | 15.6                       | 18.3                    |
| Proposed Method | 9.8                        | 12.4                    |

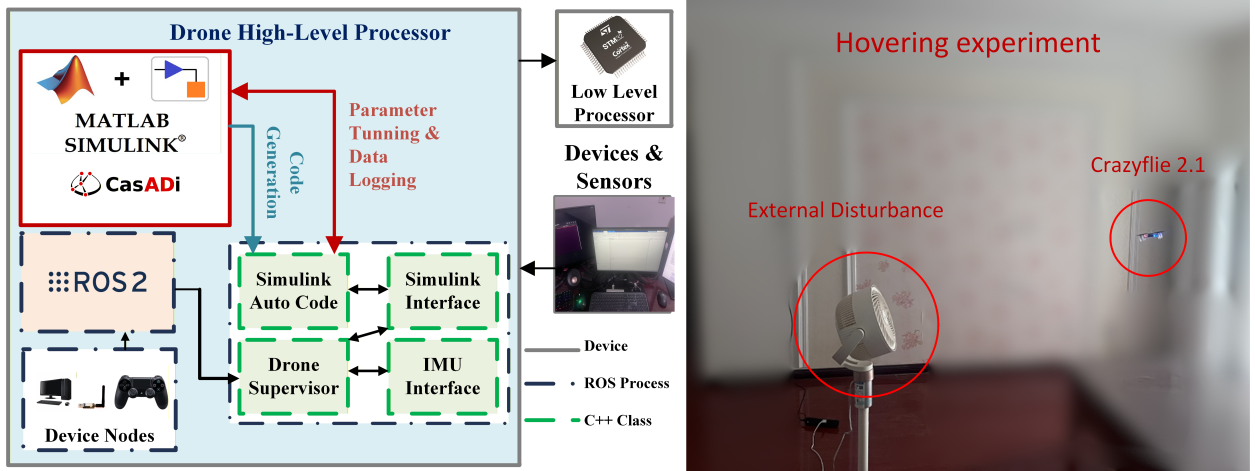


Figure 7: Simulation to hardware deployment and hovering experiments

Table 4: RMSE (m) and average computation time per iteration (ms) across different trajectories.

| Trajectory type                      | PID           | SMPC          | FT-MPC        | NN-MPC        | Proposed      |
|--------------------------------------|---------------|---------------|---------------|---------------|---------------|
| Helical (m)                          | 0.3686        | 0.2516        | 0.1530        | 0.0520        | 0.0158        |
| Spline (m)                           | 0.2000        | 0.1760        | 0.3524        | 0.1053        | 0.0183        |
| Lemniscate (m)                       | 0.1061        | 0.0701        | 0.0800        | 0.0303        | 0.0182        |
| <b>Average RMSE (m)</b>              | <b>0.2249</b> | <b>0.1659</b> | <b>0.1951</b> | <b>0.0625</b> | <b>0.0174</b> |
| <b>Average computation time (ms)</b> | <b>0.12</b>   | <b>8.45</b>   | <b>10.23</b>  | <b>15.67</b>  | <b>9.81</b>   |

hardware, including hovering experiments, with real-time results shown in Fig. 8. Comparative analysis reveals that

<sup>1</sup><https://www.bitcraze.io/>

### Hardware Experiment: State Responses During Hovering

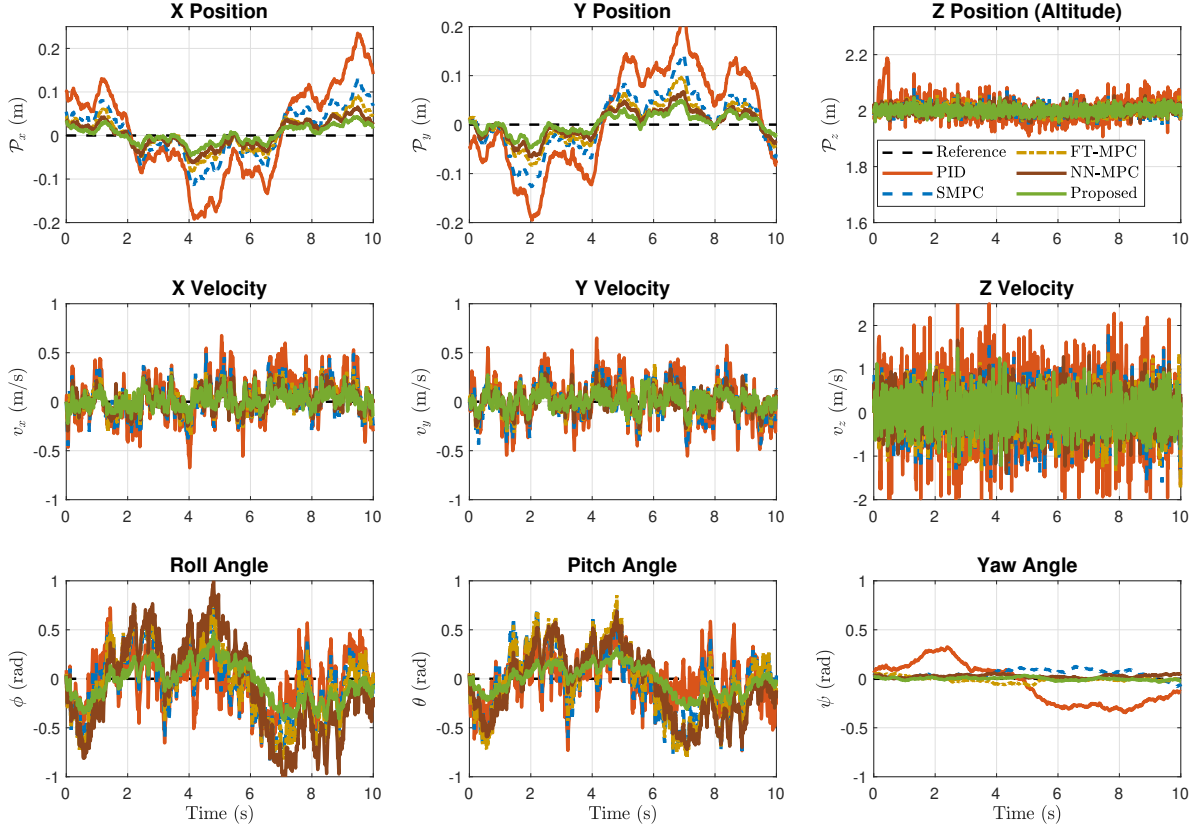


Figure 8: Real-time indoor experiments related to state response

the presented method exhibits a relatively damped response with minimal oscillations compared to other approaches. Fig. 9 shows control inputs, where our developed method produces smoother signals with reduced high-frequency content, indicating better energy efficiency and reduced motor wear. Quantitative metrics in Table 5 confirm that our approach outperforms alternatives across key performance indicators.

Table 5: Hardware experiment summary (RMSE).

| Method   | Position (m) | Attitude (rad) | Max Altitude Error (m) |
|----------|--------------|----------------|------------------------|
| PID      | 0.1552       | 0.4413         | 0.1139                 |
| SMPC     | 0.0883       | 0.4649         | 0.0763                 |
| FT-MPC   | 0.0612       | 0.4743         | 0.0599                 |
| NN-MPC   | 0.046        | 0.4621         | 0.0534                 |
| Proposed | 0.0326       | 0.2031         | 0.0506                 |

#### 4.1. Memory Consumption on STM32F405RG

Memory usage is measured via STM32CubeIDE (linker script analysis + runtime RAM monitoring) and reported in Table 6 (critical for embedded systems with limited resources):

*Memory Allocation Breakdown (Proposed Method).* : Static RAM (constants, lookup tables): 8 KB, Dynamic RAM (disturbance history: 50 samples  $\times$  12D  $\times$  4 bytes = 2.4 KB; RPI set: 12 $\times$ 12 matrix = 0.576 KB; library coefficients: 100 terms  $\times$  12D = 4.8 KB), MPC Solver RAM (CasADi workspace): 16.2 KB, Total RAM: 32 KB (16.7% of STM32F405RG's 192 KB RAM), which is well within hardware limits.

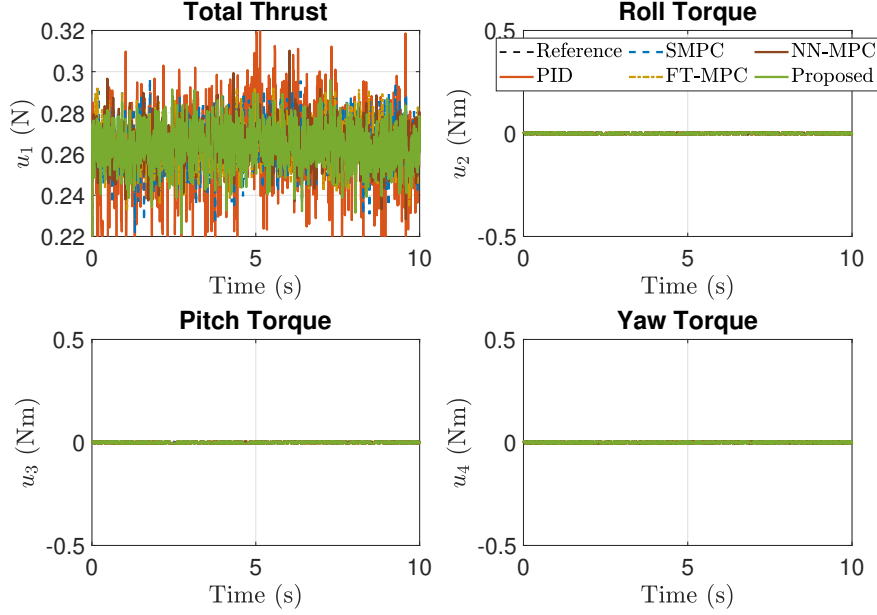


Figure 9: Control input during hovering experiment

Table 6: Embedded Memory Consumption (STM32F405RG)

| Controller      | Flash Usage (KB) | RAM Usage (KB) |
|-----------------|------------------|----------------|
| PID             | 8                | 2              |
| SMPC            | 42               | 18             |
| FT-MPC          | 56               | 24             |
| NN-MPC          | 128              | 64             |
| Proposed Method | 66               | 32             |

The disturbance history (2.4 KB) contributes minimally to overall RAM usage, and the proposed method’s memory footprint is 50% lower than NN-MPC, validating its efficiency for resource-constrained embedded platforms.

#### 4.2. Discussion, limitations and Practical Insights

The proposed PIML-MPC framework demonstrates several distinct advantages over existing control approaches. The enhanced tracking accuracy of the presented framework, evidenced by reduced RMSE error compared to both SMPC and PID controllers, is directly attributable to the learned residual dynamics model. This model captures aerodynamic and inertial effects not represented in the nominal first-principles model, improving prediction fidelity. The sparse structure of the ML-based model enables computationally efficient implementation. While the adaptive tube mechanism incurs additional computational load compared to non-adaptive approaches, the overall computational requirement remains feasible for real-time implementation on embedded hardware, as demonstrated in hardware experiments. The adaptive tube mechanism provides robustness against model uncertainty and disturbances while reducing conservatism relative to fixed-tube robust MPC formulations. This adaptation is informed by online estimates of the residual model error, allowing constraint satisfaction without excessively restricting the feasible control space. The data efficiency of the PIML approach enables effective model learning from limited flight data. The sparse regression framework, combined with physics-informed library functions, requires substantially fewer training samples than black-box NN-based alternatives to achieve comparable model accuracy. Compared with state-of-the-art learning-based MPC represented by NN-MPC, the proposed PIML sparse identification framework exhibits distinct advantages. First, benefiting from embedded physical prior knowledge and automatic sparse library construction, our method achieves satisfactory modeling accuracy with a limited number of flight samples, while black-box neural network MPC requires massive labeled datasets for training and suffers from poor generalization under unseen disturbances. Second, the learned sparse dynamics model has explicit analytical expressions that are fully interpretable,

whereas NN models are black boxes and cannot provide physical explanations for system behavior. In terms of embedded deployment on resource-constrained quadrotors, the proposed method has a much smaller memory footprint (32 KB RAM) than NN-MPC (64 KB RAM), and its average computation time is reduced by 37.4%. In addition, when combined with the real-time adaptive tube mechanism, our framework maintains robust constraint satisfaction under time-varying wind disturbances and model errors, outperforming conventional learning-based MPC without a dedicated robust design.

The proposed framework presents certain limitations that warrant consideration: (1) During the early learning phase (before sufficient flight data accumulates to refine the PIML model and RPI set), constraint tightening is intentionally conservative to ensure safety. This restricts maneuver aggressiveness to avoid constraint violations while the library and disturbance bounds adapt. The conservatism diminishes as more data is collected; (2) Discrete, periodic PIML model updates (implemented every  $T_{\text{learn}} = 5$  seconds in our experiments) may introduce minor, short-lived transients in the control signal. These transients arise from small adjustments to the nominal  $\hat{f}^d$ , and RPI set  $\mathbb{S}(k)$  during updates, though they are bounded by the tube constraints and do not compromise stability. Continuous or event-triggered update strategies may mitigate this effect by aligning updates with meaningful model improvements rather than fixed time intervals; (3) The adaptive disturbance set update requires storing a short history of realized disturbances  $\mathbf{\Lambda}_{\text{vw}}^d(k)$  to compute  $\lambda$  and  $\gamma$ . This increases memory overhead by 8 KB (for 12-dimensional disturbances stored as 32-bit floats) relative to non-adaptive robust controllers (which use static disturbance bounds). While negligible for modern embedded MCUs (e.g., STM32F405RG with 192 KB RAM), it may be a consideration for ultra-low-power platforms with limited memory; (4) The efficacy of the sparse identification process is contingent upon the completeness of the candidate function library; an overly restrictive (incomplete) or redundant (bloated) library can degrade model accuracy. Critically, this does not imply manual library selection, as our library is constructed automatically via physics-required base terms and data-driven expansion. The limitation arises from the fixed degree of nonlinearity in auto-expanded terms-extreme dynamics (e.g., high-speed aerodynamic stall) may require higher-degree terms or specialized physics terms not included in the current auto-expansion rule.

Practical implementation yielded several insights. The adaptation gain parameter requires calibration based on the anticipated disturbance spectrum to balance convergence speed and noise sensitivity.  $\lambda = 0.9$  and  $\gamma = 0.95$  are empirically calibrated via repeated simulations and Crazyflie flight tests to reconcile noise suppression and dynamic adaptivity. Slight parameter changes within  $\pm 0.03$  yield tracking RMSE fluctuations below 3.5%, and the PIML sparse learning procedure stays unaffected since  $\lambda$  and  $\gamma$  merely regulate online disturbance-bound updating rather than the sparse regression for system identification. Closed-loop stability is preserved across such mild parameter perturbations. Note that the empirical selection of  $\lambda$  and  $\gamma$  does not constitute a critical drawback for the overall framework, as these hyperparameters only govern the recursive update of disturbance bounds instead of the core PIML identification and tube MPC formulation. Owing to the low performance sensitivity against mild parameter variations, the empirically chosen values ensure reliable real-world flight performance. Analytical optimal gain solving can be explored in future research to further remove empirical tuning. A safety fallback mode, triggered by metrics indicating model degradation, can enhance operational reliability. The control energy expenditure of the proposed method was observed to be intermediate between simpler PID controllers and more computationally intensive NN-based MPC. The computational complexity of the presented algorithm scales approximately linearly with the system state dimension, suggesting applicability to larger aerial platforms without prohibitive computational cost increase. These findings indicate that the developed framework provides a viable compromise between tracking precision, robustness guarantees, and computational demand, rendering it applicable to aerial vehicles operating under practical safety and resource constraints.

## 5. Conclusion

This paper presented a PIML-based MPC framework that leverages online PIML model learning with robust control guarantees. The key contributions include: (1) a PIML identification method that provides interpretable dynamics models, (2) an online disturbance set adaptation mechanism that reduces conservatism while maintaining robustness, and (3) formal guarantees of recursive feasibility and ISS. Numerical simulations and experiments demonstrated that our approach reduces computational load compared to SMPC, FT-MPC, and NN-MPC while maintaining comparable tracking performance. The adaptive tube mechanism outperformed FT-MPC by reducing conservatism without compromising safety.

Future work will focus on extending the framework to multi-vehicle systems and investigating distributed implementation strategies for swarm applications. Additionally, we plan to explore transfer learning techniques to reduce the initial data requirements for new vehicle configurations.

## Appendix A. Proof of Lipschitz Continuity for the Quadratic Stage Cost (Assumption 7)

Assumption 7 states that the quadratic stage cost for the quadrotor MPC framework:

$$\ell_c(\mathbf{x}, \mathbf{u}) = (\mathbf{x} - \mathbf{x}_{\text{ref}})^\top \mathbf{Q}(\mathbf{x} - \mathbf{x}_{\text{ref}}) + (\mathbf{u} - \mathbf{u}_{\text{ref}})^\top \mathbf{R}(\mathbf{u} - \mathbf{u}_{\text{ref}}),$$

is Lipschitz continuous on the compact set  $\mathbb{X} \times \mathbb{U}$ . Here, the state set  $\mathbb{X} \subset \mathbb{R}^{12}$  and input set  $\mathbb{U} \subset \mathbb{R}^4$  are defined by quadrotor physical constraints, with  $\mathbf{Q} \geq 0$  (positive semi-definite) and  $\mathbf{R} > 0$  (positive definite). We prove this property below.

*Proof.* Let  $(\mathbf{x}_1, \mathbf{u}_1), (\mathbf{x}_2, \mathbf{u}_2) \in \mathbb{X} \times \mathbb{U}$  be arbitrary state-input pairs within the compact constraint set. Define:

$$\begin{aligned} \Delta \mathbf{x} &= \mathbf{x}_1 - \mathbf{x}_2, & \Delta \mathbf{u} &= \mathbf{u}_1 - \mathbf{u}_2, \\ \mathbf{x}_0 &= \mathbf{x}_{\text{ref}}, & \mathbf{u}_0 &= \mathbf{u}_{\text{ref}}. \end{aligned}$$

First, expand the difference of the stage cost:

$$\begin{aligned} \ell_c(\mathbf{x}_1, \mathbf{u}_1) - \ell_c(\mathbf{x}_2, \mathbf{u}_2) &= \underbrace{(\mathbf{x}_1 - \mathbf{x}_0)^\top \mathbf{Q}(\mathbf{x}_1 - \mathbf{x}_0) - (\mathbf{x}_2 - \mathbf{x}_0)^\top \mathbf{Q}(\mathbf{x}_2 - \mathbf{x}_0)}_{\text{State cost difference}} \\ &\quad + \underbrace{(\mathbf{u}_1 - \mathbf{u}_0)^\top \mathbf{R}(\mathbf{u}_1 - \mathbf{u}_0) - (\mathbf{u}_2 - \mathbf{u}_0)^\top \mathbf{R}(\mathbf{u}_2 - \mathbf{u}_0)}_{\text{Input cost difference}}. \end{aligned}$$

Step 1: Simplify the quadratic form difference.

Use the identity for symmetric matrices ( $M = M^\top$ ):

$$\mathbf{a}^\top M \mathbf{a} - \mathbf{b}^\top M \mathbf{b} = (\mathbf{a} - \mathbf{b})^\top M (\mathbf{a} + \mathbf{b} - 2\mathbf{c}),$$

where  $\mathbf{c}$  denotes a constant vector. For the state term, we set  $\mathbf{c} = \mathbf{x}_0$ , and for the input term  $\mathbf{c} = \mathbf{u}_0$ . Applying this identity:

$$\begin{aligned} (\mathbf{x}_1 - \mathbf{x}_0)^\top \mathbf{Q}(\mathbf{x}_1 - \mathbf{x}_0) - (\mathbf{x}_2 - \mathbf{x}_0)^\top \mathbf{Q}(\mathbf{x}_2 - \mathbf{x}_0) &= \Delta \mathbf{x}^\top \mathbf{Q}(\mathbf{x}_1 + \mathbf{x}_2 - 2\mathbf{x}_0), \\ (\mathbf{u}_1 - \mathbf{u}_0)^\top \mathbf{R}(\mathbf{u}_1 - \mathbf{u}_0) - (\mathbf{u}_2 - \mathbf{u}_0)^\top \mathbf{R}(\mathbf{u}_2 - \mathbf{u}_0) &= \Delta \mathbf{u}^\top \mathbf{R}(\mathbf{u}_1 + \mathbf{u}_2 - 2\mathbf{u}_0). \end{aligned}$$

Substitute back into the cost difference:

$$\ell_c(\mathbf{x}_1, \mathbf{u}_1) - \ell_c(\mathbf{x}_2, \mathbf{u}_2) = \Delta \mathbf{x}^\top \mathbf{Q}(\mathbf{x}_1 + \mathbf{x}_2 - 2\mathbf{x}_0) + \Delta \mathbf{u}^\top \mathbf{R}(\mathbf{u}_1 + \mathbf{u}_2 - 2\mathbf{u}_0).$$

Step 2: Apply the Cauchy-Schwarz Inequality.

For any vectors  $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$ , the Cauchy-Schwarz inequality reads:

$$|\mathbf{a}^\top \mathbf{b}| \leq \|\mathbf{a}\| \cdot \|\mathbf{b}\|.$$

Applying this to both terms:

$$\begin{aligned} |\Delta \mathbf{x}^\top \mathbf{Q}(\mathbf{x}_1 + \mathbf{x}_2 - 2\mathbf{x}_0)| &\leq \|\Delta \mathbf{x}\| \cdot \|\mathbf{Q}(\mathbf{x}_1 + \mathbf{x}_2 - 2\mathbf{x}_0)\|, \\ |\Delta \mathbf{u}^\top \mathbf{R}(\mathbf{u}_1 + \mathbf{u}_2 - 2\mathbf{u}_0)| &\leq \|\Delta \mathbf{u}\| \cdot \|\mathbf{R}(\mathbf{u}_1 + \mathbf{u}_2 - 2\mathbf{u}_0)\|. \end{aligned}$$

Step 3: Bounding terms via compactness.

Since  $\mathbb{X} \times \mathbb{U}$  is closed and bounded (compact) due to quadrotor physical constraints, the terms  $\mathbf{x}_1 + \mathbf{x}_2 - 2\mathbf{x}_0$  and  $\mathbf{u}_1 + \mathbf{u}_2 - 2\mathbf{u}_0$  are uniformly bounded. Define:

$$\begin{aligned}\mathcal{L}_x &= \sup_{\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{X}} \|\mathbf{Q}(\mathbf{x}_1 + \mathbf{x}_2 - 2\mathbf{x}_0)\|, \\ \mathcal{L}_u &= \sup_{\mathbf{u}_1, \mathbf{u}_2 \in \mathbb{U}} \|\mathbf{R}(\mathbf{u}_1 + \mathbf{u}_2 - 2\mathbf{u}_0)\|.\end{aligned}$$

Both  $\mathcal{L}_x > 0$  and  $\mathcal{L}_u > 0$  are finite constants, as the supremum of a continuous function over a compact set is bounded.

Step 4: Final Lipschitz bound.

Combining the above results yields the bound for the absolute cost difference:

$$|\ell_c(\mathbf{x}_1, \mathbf{u}_1) - \ell_c(\mathbf{x}_2, \mathbf{u}_2)| \leq \mathcal{L}_x \cdot \|\Delta \mathbf{x}\| + \mathcal{L}_u \cdot \|\Delta \mathbf{u}\|.$$

This satisfies the definition of Lipschitz continuity with Lipschitz constants  $\mathcal{L}_x$  (for states) and  $\mathcal{L}_u$  (for inputs). Therefore, the quadratic stage cost is Lipschitz continuous on  $\mathbb{X} \times \mathbb{U}$ .  $\square$

### CRediT Authorship Contribution Statement

**Tayyab Manzoor:** Writing review & editing, Project administration, Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data Curation, Visualization, Project administration, Funding acquisition. **Yasir Ali:** Writing review & editing, Visualization, Validation, Formal analysis, Funding acquisition. **Yuanqing Xia:** Writing review & editing, Supervision, Visualization, Funding acquisition, Project administration. **Lijie You:** Writing review & editing, Visualization, Validation, Project administration. **Yan Wang:** Writing review & editing, Visualization, Validation, Project administration.

### Acknowledgment

This work was supported in part by the Beijing Natural Science Foundation Haidian Original Innovation Joint Fund Project under Grant L252035, in part by the National Natural Science Foundation of China under Grant U25A20460, in part by the National Natural Science Foundation of China for International Young Scientists under Grant W2533176, in part by the Beijing Municipal Natural Science Foundation for Foreign Scholars under Grant IS25064, Henan Province Key International Science and Technology Cooperation Project under Grant 251111520400, in part by the Joint Fund Project of Shandong Provincial Natural Science Foundation under Grant ZR2025LZH001, in part by the Henan Provincial Talent Program under Grant 264000510006 and in part by the Henan Postdoctoral Foundation under Grant HN2026058.

### References

- [1] L.-X. Xu, Reinforcement learning-based distributed optimal formation tracking control with obstacle avoidance for quadrotor UAVs, *Aerospace Science and Technology* 173 (2026) 111789. doi:10.1016/j.ast.2026.111789.
- [2] K. Tian, L.-J. Hu, C. Bai, H.-P. Ren, Adaptive model predictive control with extended state observer for unmanned aerial vehicle trajectory tracking, *Aerospace Science and Technology* 168 (2026) 111085. doi:10.1016/j.ast.2025.111085.
- [3] X. Cheng, Y. Wang, J. Zhao, N. Liu, Z. Cai, Data-driven adaptive hybrid-based control for some tiltrotor UAVs in the low-speed flight, *Aerospace Science and Technology* 170 (2026) 111513. doi:10.1016/j.ast.2025.111513.
- [4] M. Osman, Y. Xia, M. Mahdi, T. Manzoor, G. E. M. Abro, A. H. Bajodah, Physics-informed sparse reinforcement learning for hybrid VTOL UAV control: HILS verification and tethered hover benchmarking, *Aerospace Science and Technology* 172 (2026) 111646. doi:10.1016/j.ast.2026.111646.

- [5] H. Ye, J. Cao, X. Yang, S. Shao, Safety-critical model predictive control for quadcopter UAV subject to wind disturbances and measurement errors in confined environments, *Aerospace Science and Technology* 173 (2026) 111790. doi:10.1016/j.ast.2026.111790.
- [6] T. Manzoor, H. Pei, Z. Sun, Z. Cheng, Model predictive control technique for ducted fan aerial vehicles using physics-informed machine learning, *Drones* 7 (1) (2023). doi:10.3390/drones7010004.
- [7] J. Drgoa, T. X. Nghiem, T. Beckers, M. Fazlyab, E. Mallada, C. Jones, D. Vrabie, S. L. Brunton, R. Findeisen, Safe physics-informed machine learning for dynamics and control, in: 2025 American Control Conference (ACC), 2025, pp. 591–606. doi:10.23919/ACC63710.2025.11107836.
- [8] C. Kong, B. Wang, L. He, Q. Jiang, C. Liu, ANN-MPC: An adaptive neural network-based nonlinear model predictive control method for robust trajectory tracking of quadrotors, *IEEE Robotics and Automation Letters* 10 (10) (2025) 10966–10973. doi:10.1109/LRA.2025.3607272.
- [9] A. K. Kamath, S. G. Anavatti, M. Feroskhan, A physics-informed neural network approach to augmented dynamics visual servoing of multirotors, *IEEE Transactions on Cybernetics* 54 (11) (2024) 6319–6332. doi:10.1109/TCYB.2024.3413072.
- [10] M. Ghanifar, A. Ali Nikkhah, M. Kamzan, M. Teshnehlal, M. Tayefi, Online trajectory regeneration for multirotors via a proportional-derivative physics-informed neural network, *IEEE Access* 13 (2025) 145168–145190. doi:10.1109/ACCESS.2025.3598862.
- [11] S. H. Hong, J. Ou, Y. Wang, Physics-guided neural network and gpu-accelerated nonlinear model predictive control for quadcopter, *Neural Computing and Applications* 35 (1) (2023) 393–413. doi:https://doi.org/10.1007/s00521-022-07783-4.
- [12] A. Saviolo, G. Li, G. Loianno, Physics-inspired temporal learning of quadrotor dynamics for accurate model predictive trajectory tracking, *IEEE Robotics and Automation Letters* 7 (4) (2022) 10256–10263. doi:10.1109/LRA.2022.3192609.
- [13] C. A. Osorio Quero, J. Martinez-Carranza, Physics-informed machine learning for UAV control, in: 2024 21st International Conference on Electrical Engineering, Computing Science and Automatic Control (CCE), 2024, pp. 1–6. doi:10.1109/CCE62852.2024.10770871.
- [14] R. Abdulkadirov, P. Lyakhov, D. Butusov, N. Nagornov, D. Kalita, Physics-aware machine learning approach for high-precision quadcopter dynamics modeling, *Drones* 9 (3) (2025). doi:10.3390/drones9030187.
- [15] P. Aryan, S. Panichella, Physics-informed machine learning for precision unmanned aerial vehicle control: Adaptive transformers with safety guarantees, *Engineering Applications of Artificial Intelligence* 172 (2026) 114379. doi:10.1016/j.engappai.2026.114379.
- [16] T. Manzoor, H. Pei, Y. Xia, Z. Sun, Y. Ali, Compound learning-based model predictive control approach for ducted-fan aerial vehicles, *IEEE Transactions on Neural Networks and Learning Systems* 36 (5) (2025) 9395–9407. doi:10.1109/TNNLS.2024.3422401.
- [17] J. Wang, M. Fazlyab, Actorcritic physics-informed neural lyapunov control, *IEEE Control Systems Letters* 8 (2024) 1751–1756. doi:10.1109/LCSYS.2024.3416235.
- [18] A. Saviolo, J. Frey, A. Rathod, M. Diehl, G. Loianno, Active learning of discrete-time dynamics for uncertainty-aware model predictive control, *IEEE Transactions on Robotics* 40 (2024) 1273–1291. doi:10.1109/TR0.2023.3339543.
- [19] A. Tagliabue, J. P. How, Efficient deep learning of robust policies from MPC using imitation and tube-guided data augmentation, *IEEE Transactions on Robotics* 40 (2024) 4301–4321. doi:10.1109/TR0.2024.3431988.

- [20] Y. Li, X. Yang, X. Zheng, Disturbance-learning-based robust model predictive control for attitude tracking of small aircraft, *IEEE Transactions on Industrial Electronics* (2025) 1–11 [doi:10.1109/TIE.2025.3536558](https://doi.org/10.1109/TIE.2025.3536558).
- [21] M. Osman, Y. Xia, M. Mahdi, T. Manzoor, A. H. Bajodah, A. Ali, A. Ali, A. Ahmed, An adaptive SINDy-lyapunov model predictive control framework for dual-system VTOL UAVs, *International Journal of Robust and Nonlinear Control* 36 (5) (2026) 2388–2417. [doi:10.1002/rnc.70272](https://doi.org/10.1002/rnc.70272).
- [22] J. D. Lee, Y. Kim, Y. Kim, H. Bang, Sparse identification of nonlinear dynamics-based model predictive control for multirotor collision avoidance, *IET Control Theory & Applications* 19 (1) (2025) e70049. [doi:https://doi.org/10.1049/cth2.70049](https://doi.org/10.1049/cth2.70049).
- [23] A. Rashad, H. Ghorab, M. Othman, A. El-Badawy, Data-driven MPC for quadrotor UAVs using SINDy, in: 2024 34th International Conference on Computer Theory and Applications (ICCTA), 2024, pp. 165–170. [doi:10.1109/ICCTA64612.2024.10974872](https://doi.org/10.1109/ICCTA64612.2024.10974872).
- [24] H. Sun, L. Dai, P. Wang, An efficient moving obstacle avoidance scheme for UAVs via output feedback robust mpc, *IEEE Transactions on Aerospace and Electronic Systems* 60 (5) (2024) 6199–6212. [doi:10.1109/TAES.2024.3401094](https://doi.org/10.1109/TAES.2024.3401094).
- [25] M. Heinlein, S. Subramanian, S. Lucia, Robust model predictive control exploiting monotonicity properties, *IEEE Transactions on Automatic Control* 70 (9) (2025) 6260–6267. [doi:10.1109/TAC.2025.3558137](https://doi.org/10.1109/TAC.2025.3558137).
- [26] W. Qin, W. Tang, W. Xia, Robust fault-tolerant control based on set-membership estimation and tube MPC, *IEEE Transactions on Automation Science and Engineering* 22 (2025) 19499–19510. [doi:10.1109/TASE.2025.3594583](https://doi.org/10.1109/TASE.2025.3594583).
- [27] L. Chen, Y. Liu, Z. Sun, L. Yang, Q. Yan, A tube-based mpc method for path tracking of autonomous vehicles with disturbances at handling limits, *IEEE Transactions on Vehicular Technology* (2025) 1–15 [doi:10.1109/TVT.2025.3620016](https://doi.org/10.1109/TVT.2025.3620016).
- [28] H. Lin, L. Zhang, W. Xie, Trajectory tracking control of quadrotor UAV with tube-based model predictive control, in: 2025 37th Chinese Control and Decision Conference (CCDC), 2025, pp. 3197–3202. [doi:10.1109/CCDC65474.2025.11090705](https://doi.org/10.1109/CCDC65474.2025.11090705).
- [29] P. K. Mishra, M. V. Gasparino, G. Chowdhary, Deep model predictive control with stability guarantees, *IEEE Transactions on Automatic Control* 70 (8) (2025) 5460–5467. [doi:10.1109/TAC.2025.3550072](https://doi.org/10.1109/TAC.2025.3550072).
- [30] D. Limon, T. Alamo, D. M. Raimondo, D. M. de la Peña, J. M. Bravo, A. Ferramosca, E. F. Camacho, Input-to-State Stability: A Unifying Framework for Robust Model Predictive Control, Springer Berlin Heidelberg, Berlin, Heidelberg, 2009, pp. 1–26. [doi:10.1007/978-3-642-01094-1\\_1](https://doi.org/10.1007/978-3-642-01094-1_1).
- [31] E. D. Sontag, Input to State Stability: Basic Concepts and Results, Springer Berlin Heidelberg, Berlin, Heidelberg, 2008, pp. 163–220. [doi:10.1007/978-3-540-77653-6\\_3](https://doi.org/10.1007/978-3-540-77653-6_3).
- [32] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, M. Diehl, CasADi – A software framework for nonlinear optimization and optimal control, *Mathematical Programming Computation* 11 (1) (2019) 1–36. [doi:10.1007/s12532-018-0139-4](https://doi.org/10.1007/s12532-018-0139-4).
- [33] A. Wächter, L. T. Biegler, On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming, *Mathematical Programming* 106 (2006) 25–57. [doi:10.1007/s10107-004-0559-y](https://doi.org/10.1007/s10107-004-0559-y).